합성곱 신경망 (CNN)

합성곱 계층 · 풀링 계층 · im2col · SimpleConvNet · LeNet과 AlexNet

이번 주의 핵심 질문

지금까지의 신경망은 MNIST의 28×28 이미지를 길이 784의 벡터로 **펼쳐서** 입력하였다. 그러면 "이 픽셀의 바로 위와 바로 옆에 어떤 픽셀이 있는가"라는 **공간 정보**를 신경망이 직접 활용하지 못한다. 이미지의 모양을 그대로 유지한 채 학습하는 방법은 없을까? 그리고 이미지의 어느 위치에 있든 같은 모양(예: 세로 획)을 같은 방법으로 알아보려면 어떻게 해야 할까?

학습목표

이 주차가 끝나면 다음을 설명하고 직접 계산할 수 있어야 한다.

1. 완전연결 계층과 비교하여 합성곱 계층이 이미지에 적합한 이유(공간 정보, 가중치 공유)를 설명한다.
2. 합성곱 연산을 손으로 계산하고, 패딩과 스트라이드에 따른 출력 크기를 구한다.
3. 여러 채널, 여러 필터, 미니배치에 대한 합성곱을 4차원 배열의 shape으로 설명한다.
4. 최대 풀링의 계산과 그 특징을 설명한다.
5. im2col을 이용하여 합성곱을 하나의 행렬곱(Affine 계층)으로 바꾸고, 그 역전파를 유도한다.
6. 합성곱 계층과 풀링 계층으로 CNN을 구성하여 MNIST를 학습시키고, 학습된 필터를 해석한다.
7. LeNet과 AlexNet의 구조와 차이를 설명한다.

1 전체 구조

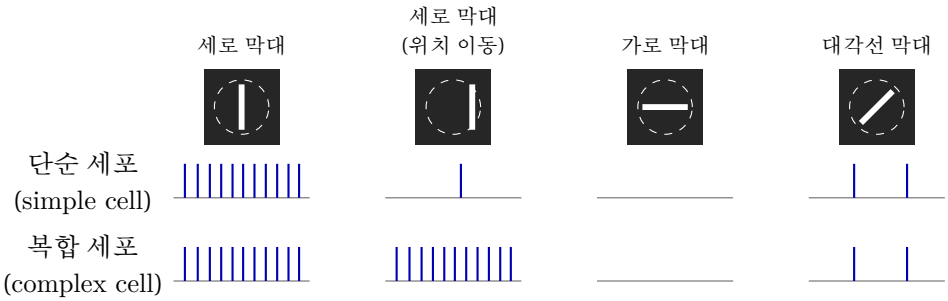
합성곱 신경망(Convolutional Neural Network, CNN)은 이미지 인식과 음성 인식 등 다양한 곳에서 쓰이는 신경망이다. 지금까지 본 신경망과 마찬가지로 계층을 조합하여 만들지만, **합성곱 계층(convolutional layer)**과 **풀링 계층(pooling layer)**이라는 새로운 계층이 등장한다. 이 두 계층은 수학에서 먼저 나온 것이 아니라, **동물의 시각 피질에 대한 실험**에서 출발하였다.

출발점: 고양이의 시각 피질 (Hubel과 Wiesel, 1959–1962). Hubel과 Wiesel은 마취한 고양이에게 스크린으로 여러 무늬를 보여 주면서, 뇌의 **1차 시각 피질**에 꽂은 전극으로 뉴런 **하나**의 반응(전기 신호, spike)을 기록하였다. 점 무늬로는 반응이 거의 없었는데, 슬라이드를 갈아 끼우다가 슬라이드 가장자리가 만든 **비스듬한 선**이 지나가는 순간 뉴런이 격렬하게 반응했다는 일화가 유명하다.



(a) 실험의 개념도. 고양이가 스크린의 막대 무늬를 보는 동안 시각 피질의 뉴런 하나의 반응을 기록한다.

막대의 방향과 위치를 바꾸어 가며 기록한 결과, 성질이 다른 두 종류의 뉴런을 찾았다. 점선 원은 그 뉴런의 **수용 영역(receptive field)**, 즉 그 뉴런이 담당하는 시야의 좁은 영역이다.

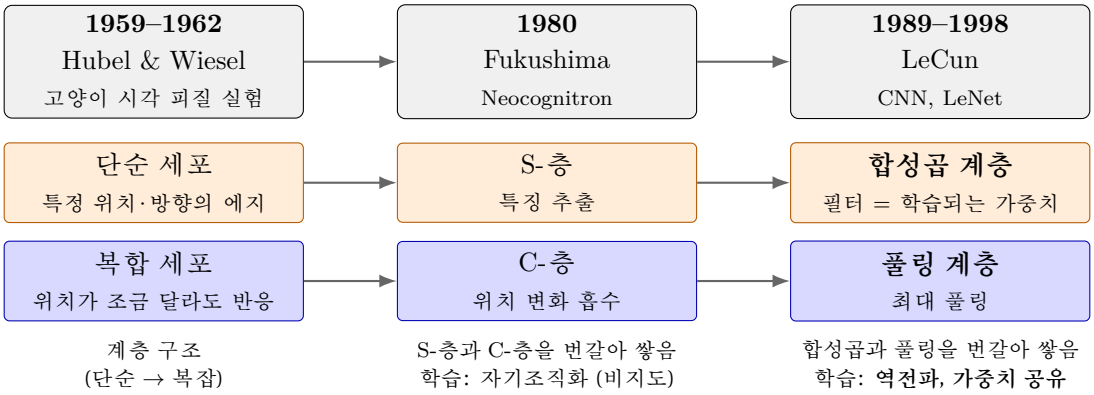


(b) 막대를 보여 줄 때의 spike (세로 선 하나가 신호 하나). 이 두 뉴런은 모두 세로 방향을 좋아한다.

- **단순 세포:** 자기 수용 영역 안의 **특정 위치에 특정 방향**의 에지가 있을 때만 반응한다. 방향이 같아도 위치가 조금 어긋나면 반응하지 않는다. 2.2절에서 볼 **Sobel 같은 에지 필터**와 같은 일을 한다.
- **복합 세포:** 방향만 맞으면 수용 영역 안에서 **위치가 조금 달라도** 반응한다. 여러 단순 세포의 출력을 모아 그중 하나라도 반응하면 반응하는 것으로 설명된다. 이것은 **최대 풀링**과 같은 일이다.

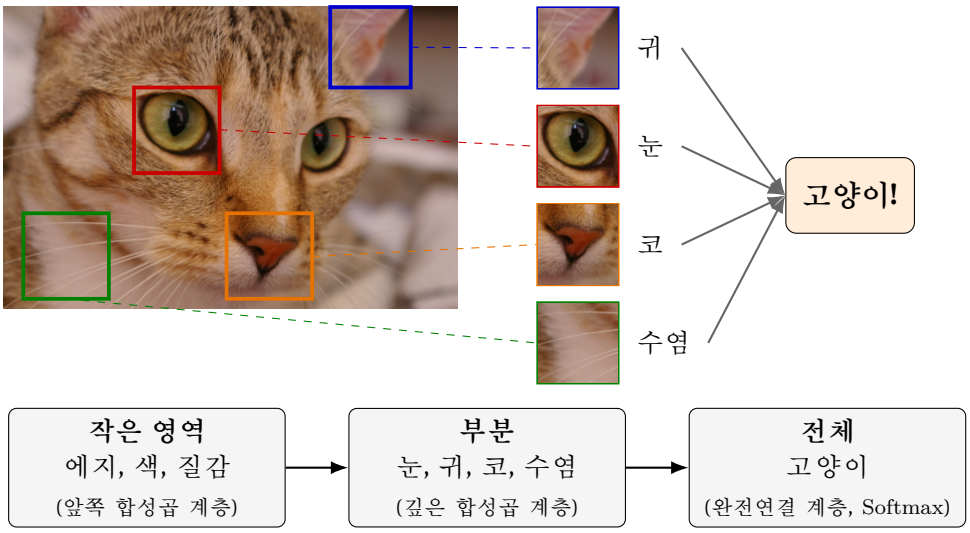
두 세포는 따로 있는 것이 아니라 **단계로 이어져** 있다. 단순 세포는 앞 단계(망막과 시상)에서 온 점 모양의 신호를 일렬로 모아 선분을 만들고, 복합 세포는 방향이 같고 위치가 조금씩 다른 단순 세포 여러 개를 모은다. 즉 **앞 단계의 출력을 모아 조금 더 복잡하거나 위치에 덜 민감한 특징을 만든다**는 것이 Hubel과 Wiesel이 제안한 **계층 모델**이다. 이 쌓기는 1차 시각 피질(V1) 다음 영역 (V2, V4, 하측두엽)에서도 계속되어, 뒤로 갈수록 보는 범위가 넓어지고 모서리, 모양, 얼굴과 사물처럼 더 복잡한 대상에 반응한다. Hubel과 Wiesel은 이 연구로 1981년 노벨 생리의학상을 받았다.

시각 피질에서 CNN으로. Fukushima는 이 발견을 그대로 신경망으로 옮긴 **Neocognitron** (1980)을 만들었다. 단순 세포에 해당하는 S-층과 복합 세포에 해당하는 C-층을 번갈아 쌓아, 위치가 조금 달라도 손글씨 숫자를 알아보았다. 다만 가중치는 역전파가 아닌 자기조직화(비지도) 규칙으로 정하였다. LeCun은 여기에 **역전파**와 **가중치 공유**를 결합하여 지금의 CNN을 만들었고 (1989), 이것이 우편번호와 수표의 손글씨를 읽는 **LeNet** (1998, 7.1절)이 되었다. 이후 GPU와 대규모 데이터 (ImageNet)를 만나 2012년 **AlexNet** (7.2절)으로 딥러닝의 시대를 열었다.



CNN의 계보. 같은 행의 상자는 같은 역할을 한다: 단순 세포 → S-층 → 합성곱 계층, 복합 세포 → C-층 → 풀링 계층.

고양이 사진을 볼 때 우리는 13만 개의 픽셀을 한꺼번에 비교하지 않는다. 먼저 눈, 귀, 코, 수염 같은 **부분**을 알아보고, 그 부분들이 제자리에 모여 있으니 고양이라고 판단한다. 이때 각 부분은 사진 전체가 아니라 **그 주변의 좁은 영역**만 보면 알아볼 수 있다.



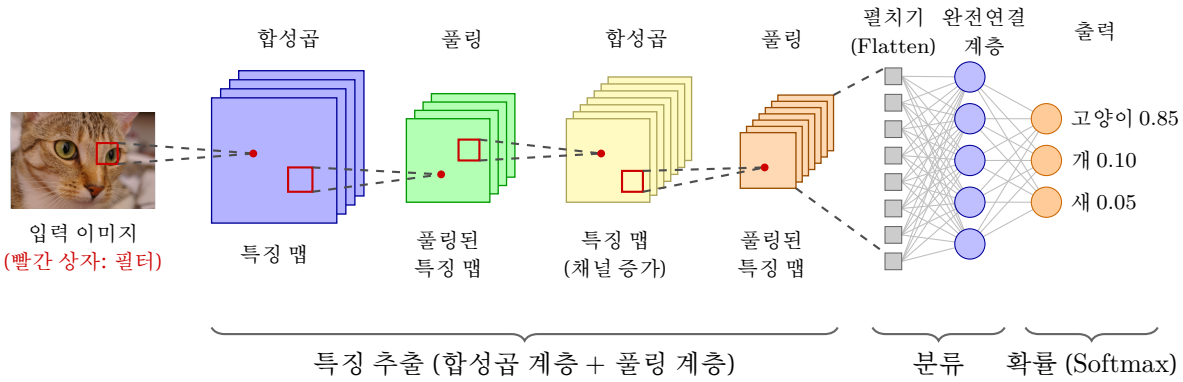
CNN은 이 과정을 모델링 한다.

- **좁은 영역씩 본다:** 합성곱 계층의 출력 하나는 입력의 작은 영역 (예: 3×3, 5×5)에만 연결된다.
- **같은 검출기를 어디에나 쓴다:** "눈 검출기"는 왼쪽 눈에도 오른쪽 눈에도 쓸 수 있어야 한다. 그래서 하나의 필터를 이미지의 모든 위치로 밀면서 적용한다.
- **단순한 것에서 복잡한 것으로:** 앞쪽 계층은 에지나 색 같은 단순한 무늬를 찾고, 뒤쪽 계층은 앞에서 찾은 무늬를 조합하여 눈, 귀 같은 부분을, 마지막의 완전연결 계층은 부분들을 모아 "고양이"를 판단한다.

이 세 가지가 이번 주에 배울 합성곱 계층의 국소 연결, 가중치 공유, 그리고 계층을 쌓는 구조이다.

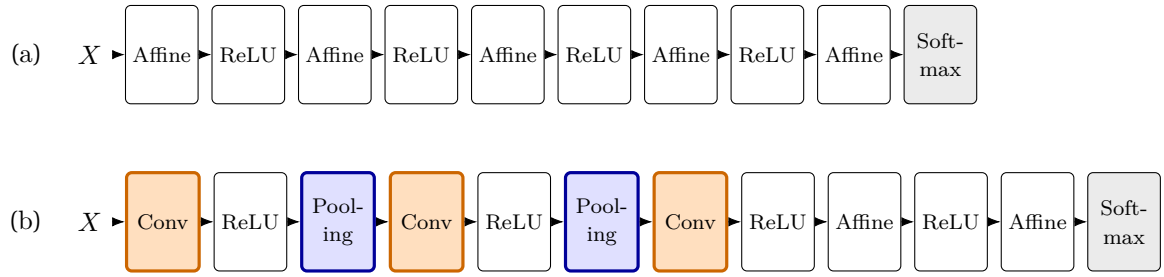
이것을 신경망의 모양으로 그리면 다음과 같다. 입력 이미지의 작은 영역이 필터(빨간 상자, 가중치)와 곱해져 합이 계산되고(합성곱) 그 값이 다음 특징 맵의 한 점이 되고, 합성곱과 풀링을 거듭할 때마다 특징 맵은 작아지는 대신 개수(채널)가 많아진다. 이때 각각의 필터는 각 특징을 추출하는

가중치 행렬이다. 마지막 특징 맵을 한 줄로 펼친 뒤에는 지금까지 배운 완전연결 계층과 Softmax로 분류한다.



전형적인 CNN의 구조. 앞부분은 특징을 추출하고, 뒷부분은 지금까지의 신경망으로 분류한다.

완전연결신경망(FNN)은 인접하는 계층의 모든 node가 연결되어 있었다. 완전연결 계층을 Affine 계층이라는 이름으로 구현하였다.



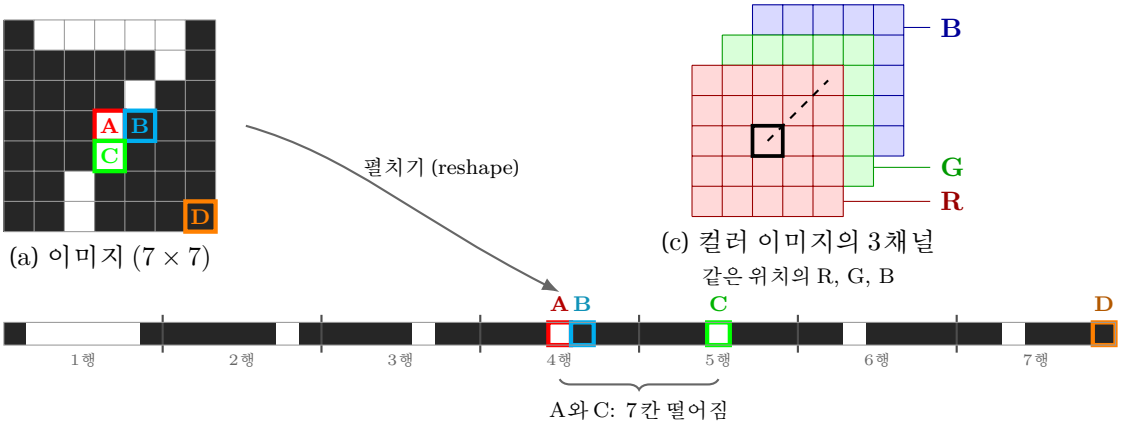
(a) 완전연결 계층(Affine)으로 이루어진 신경망. (b) CNN의 예: 주황색이 합성곱 계층(Conv), 파란색이 풀링 계층(Pooling)이다.

CNN에서는 "Affine → ReLU" 대신 "Conv → ReLU → (Pooling)"의 흐름이 이어진다(풀링 계층은 생략하기도 한다). 출력에 가까운 층에서는 지금까지의 "Affine → ReLU"를 쓰고, 마지막은 "Affine → Softmax"이다. 즉 CNN은 앞부분에서 이미지의 특징을 뽑고, 뒷부분은 지금까지의 신경망으로 분류하는 구조이다.

2 합성곱 계층

2.1 완전연결 계층의 문제점

완전연결 계층의 문제점은 **데이터의 형상이 무시된다**는 것이다. MNIST 이미지는 세로 28, 가로 28, 채널 1 (흑백)인 3차원 데이터 $(1, 28, 28)$이지만, Affine 계층에 넣으려면 784개의 값을 한 줄로 펼쳐야 했다. 이미지에서는 공간적으로 가까운 픽셀끼리 값이 비슷하고, RGB의 세 채널은 서로 밀접하게 관련되어 있으며, 떨어진 픽셀끼리는 관련이 적다. 펼쳐도 값과 순서는 그대로 남아 있지만 (reshape로 되돌릴 수 있다), 완전연결 계층은 이런 **3차원 모양에 담긴 패턴**을 구조적으로 활용하지 못한다. 완전연결 계층은 모든 입력을 같은 차원의 값으로 취급하므로, 모든 이미지의 픽셀을 같은 순서로 섞어도 표현할 수 있는 함수는 달라지지 않는다. 즉 어느 픽셀이 어느 픽셀의 이웃인지에 대한 가정이 들어 있지 않다.



(a) 이미지 (7×7)

(c) 컬러 이미지의 3채널
같은 위치의 R, G, B

(b) 한 줄로 펼친 벡터 (49)

이미지를 펼치면 드러나지 않는 이웃 관계. (a)에서 A의 바로 옆 B와 바로 아래 C는 모두 A의 이웃이지만, (b)에서 B만 옆에 남고 C는 한 행의 길이만큼 떨어진다.

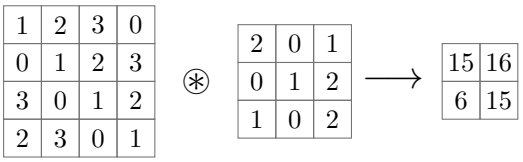
그림의 (a)에서 A와 그 이웃 B, C는 서로 가까워 값이 비슷하거나 함께 변하고 (여기서는 A와 C가 같은 획), 멀리 떨어진 D는 A와 관련이 적다. 그러나 (b)처럼 한 줄로 펼치면 A의 오른쪽 이웃 B는 바로 옆에 남지만 아래 이웃 C는 7칸 (MNIST라면 28칸) 떨어지고, 벡터만 보아서는 C가 A의 이웃이라는 것을 알 수 없다. (c)의 컬러 이미지에서는 같은 위치의 R, G, B 세 값이 한 점의 색을 함께 이루므로 밀접하게 관련되어 있다. 완전연결 계층은 이런 관계를 모르는 채로 784개의 값을 똑같이 다루어, 이웃 관계를 처음부터 데이터로 배워야 한다.

합성곱 계층은 이 **형상을 유지**한다. 이미지를 3차원 데이터로 입력받고, 다음 계층에도 3차원 데이터로 전달한다. CNN에서 합성곱 계층의 입출력 데이터를 **특징 맵(feature map)**이라 한다. 입력 데이터를 **입력 특징 맵**, 출력 데이터를 **출력 특징 맵**이라 부른다.

2.2 합성곱 연산

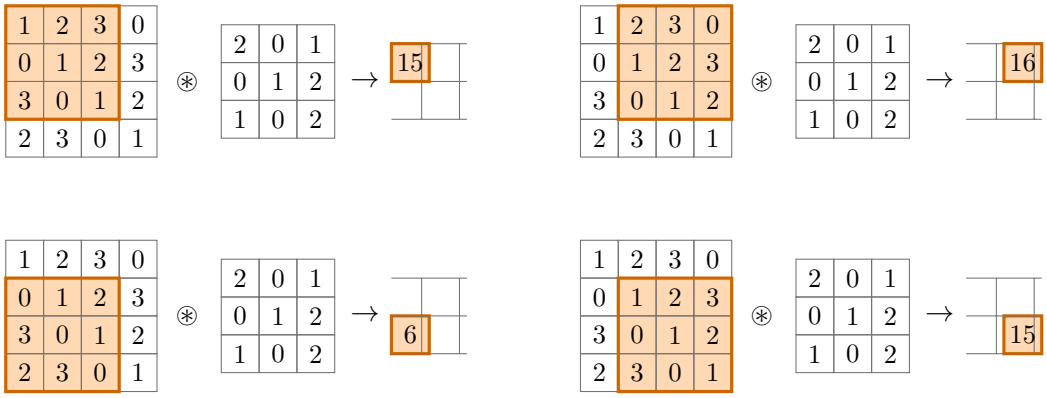
합성곱 계층에서는 **합성곱 연산**을 한다. 이미지 처리에서 말하는 **필터 연산**이다.

먼저 채널이 하나인 경우를 보자. 입력은 $(4, 4)$, 필터는 $(3, 3)$이다. 필터를 **커널(kernel)**이라고도 한다.



입력 데이터 필터 출력

합성곱 연산은 필터의 **윈도우(window)**를 일정 간격으로 이동해 가며 입력 데이터에 적용한다. 각 위치에서 입력과 필터의 **대응하는 원소끼리 곱한 뒤 모두 더한다** (곱셈-누산). 수학의 합성곱과 달리 필터를 뒤집지 않으므로 엄밀히는 상호상관(cross-correlation)이지만, 딥러닝에서는 관례적으로 합성곱이라 부른다. 그 결과를 출력의 해당 위치에 저장한다. 왼쪽 위부터 오른쪽 아래까지 모든 위치에서 이 계산을 반복한다.



합성곱 연산의 계산 순서. 주황색 윈도우가 왼쪽 위에서 오른쪽 아래로 이동한다.

예를 들어 첫 번째 위치에서는

$$1 \cdot 2 + 2 \cdot 0 + 3 \cdot 1 + 0 \cdot 0 + 1 \cdot 1 + 2 \cdot 2 + 3 \cdot 1 + 0 \cdot 0 + 1 \cdot 2 = 15$$

이다. 식으로 쓰면, 입력 x와 필터 f $(FH \times FW)$에 대하여 출력의 (i, j) 원소는

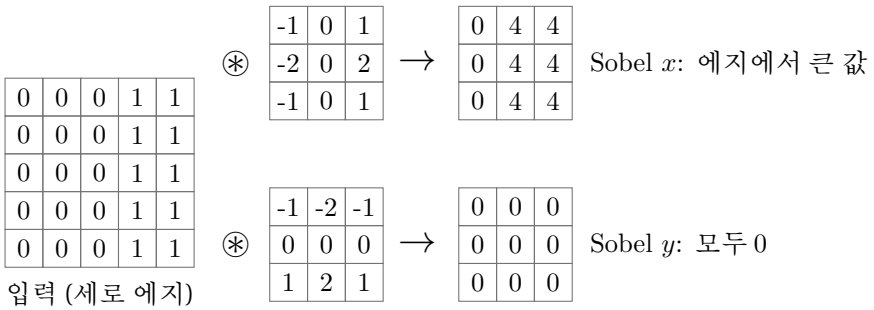
$$y_{ij} = \sum_{p=0}^{FH-1} \sum_{q=0}^{FW-1} x_{i+p,\, j+q}\, f_{pq}$$

이다.

사람이 설계한 필터: 이미지 처리에서의 특징 추출. 방금 본 계산, 즉 3×3이나 5×5 크기의 작은 행렬 (**필터**)을 이미지 위로 한 칸씩 밀면서 대응하는 원소끼리 곱해 모두 더하는 방법은 CNN이 나오기 전부터 이미지 처리에서 특징을 뽑는 데 써 왔다. 어떤 특징을 뽑을지는 필터의 값이 정하고, 그 값은 사람이 목적에 맞게 설계하였다. 대표적인 필터는 다음과 같다.

필터	값	하는 일
평균 (blur)	$\frac{1}{25}\begin{pmatrix}1 & \cdots & 1\\ \vdots & \ddots & \vdots\\ 1 & \cdots & 1\end{pmatrix}_{5\times5}$	주변 25개 픽셀의 평균. 잡음과 세부가 사라지고 흐려진다.
선명하게 (sharpen)	$\begin{pmatrix}0 & -1 & 0\\ -1 & 5 & -1\\ 0 & -1 & 0\end{pmatrix}$	가운데 픽셀과 이웃의 차이를 키운다. 윤곽이 또렷해진다.
Sobel x	$\begin{pmatrix}-1 & 0 & 1\\ -2 & 0 & 2\\ -1 & 0 & 1\end{pmatrix}$	오른쪽 열 − 왼쪽 열: 가로 방향 밝기 변화, 즉 **세로 에지**에 반응한다.
Sobel y	$\begin{pmatrix}-1 & -2 & -1\\ 0 & 0 & 0\\ 1 & 2 & 1\end{pmatrix}$	아래 행 − 위 행: 세로 방향 밝기 변화, 즉 **가로 에지**에 반응한다.
Laplacian	$\begin{pmatrix}0 & 1 & 0\\ 1 & -4 & 1\\ 0 & 1 & 0\end{pmatrix}$	이웃의 합 − 4× 가운데 (2계 미분의 근사): 방향에 관계없이 에지에 반응한다.
엠보싱 (emboss)	$\begin{pmatrix}-2 & -1 & 0\\ -1 & 1 & 1\\ 0 & 1 & 2\end{pmatrix}$	대각선 방향의 밝기 변화를 더해 돋을새김처럼 보이게 한다.

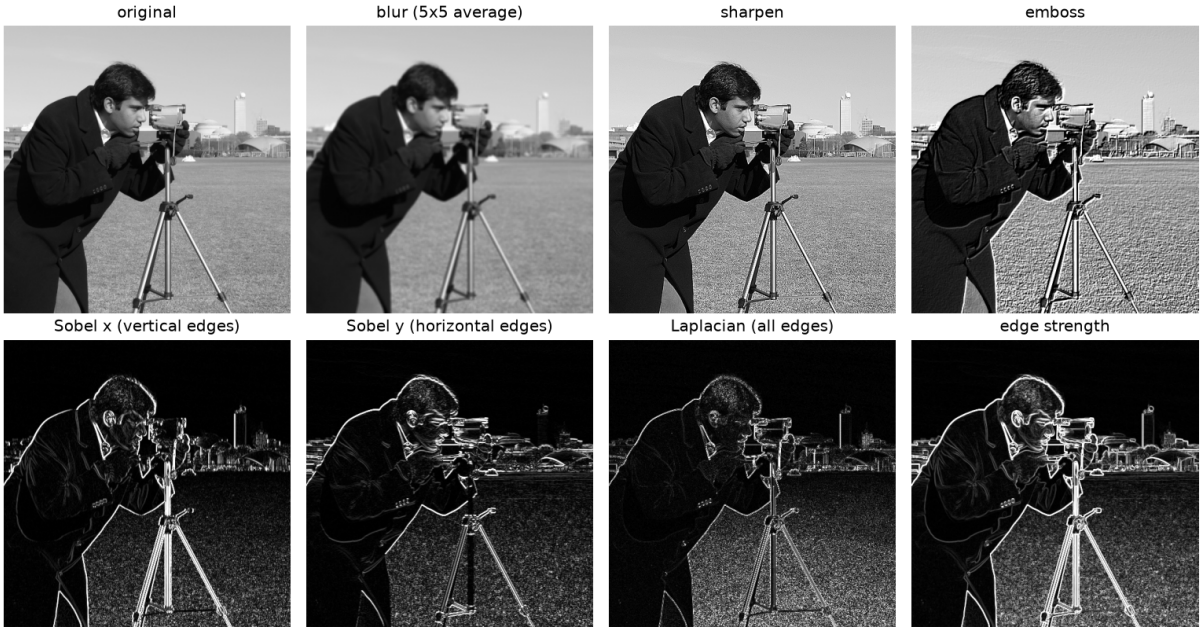
왜 이런 필터가 에지에 반응할까? 왼쪽 절반이 0(검정), 오른쪽 절반이 1(흰색)인 5×5 이미지에 Sobel 필터를 적용해 보자.



Sobel 필터의 원소를 모두 더하면 0이므로, 밝기가 고른 곳에서는 출력이 0이다. Sobel x는 창 안에 0과 1의 경계가 있는 위치에서만 $(0\cdot(-1)+1\cdot1)(1+2+1)=4$를 내고, 같은 이미지에 Sobel y를 적용하면 위아래로는 밝기 변화가 없으므로 모두 0이다. 즉 출력은 **그 위치의 이미지 조각이 필터의 무늬와 얼마나 닮았는가**를 나타낸다. 필터의 무늬(왼쪽은 음수, 오른쪽은 양수)와 닮은 곳일수록 곱의 합이 크다.

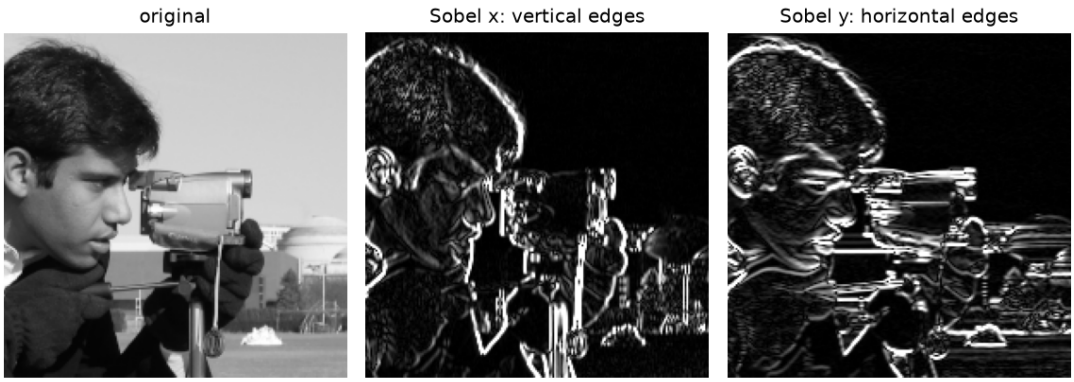
실제 흑백 사진 (512×512)에 위의 필터들을 적용하면 다음과 같다 (`classic_filters.py`). 에지 필터의 출력은 양수와 음수가 모두 나오므로 절댓값을 그렸다 (밝을수록 강한 에지). 마지막 그림은 두 Sobel 출력을 합친 **에지의 세기**이다. 같은 위치 (i,j)에서 Sobel x의 출력을 g_x, Sobel y의 출력을

g_y 라 하면 (각각 가로 방향과 세로 방향의 밝기 변화량), 그 위치의 에지의 세기는 $\sqrt{g_x^2 + g_y^2}$ 이다. 두 방향의 변화를 벡터 (g_x, g_y) 로 보고 그 길이를 잰 것이므로, 에지가 어느 방향으로 뻗어 있든 밝게 나타난다.



사람이 설계한 필터를 흑백 사진에 적용한 결과. 위: 원본, 평균, 선명하게, 엠보싱. 아래: Sobel x, Sobel y, Laplacian, 에지의 세기.

두 Sobel 필터의 차이는 확대해 보면 더 분명하다. Sobel x 에서는 삼각대 다리와 얼굴의 옆선처럼 **세로로 뻗은 경계**가 밝고, Sobel y 에서는 카메라의 윗면과 아랫면, 지평선처럼 **가로로 뻗은 경계**가 밝다. 필터 하나가 이미지에서 한 종류의 특징만 골라내는 것이다.



카메라 부분 확대. 같은 위치에서도 필터에 따라 뽑히는 에지의 방향이 다르다.

사람이 설계한 필터는 지금 어디에 쓰이는가

딥러닝이 대신하게 된 것은 "이것이 고양이인가" 같은 복잡한 인식이고, 단순하고 목적이 분명한 처리에는 지금도 고정된 필터가 쓰인다.

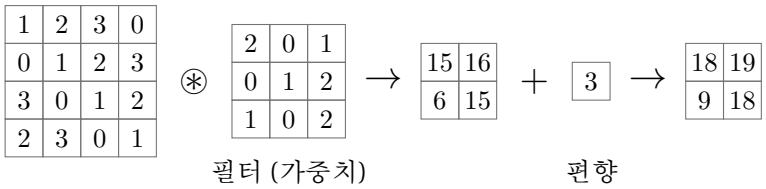
- **사진·영상 편집과 카메라:** 편집 앱의 "흐림 효과"와 "선명하게", 카메라 내부의 잡음 제거와 윤곽 보정, 이미지를 줄이기 전에 블러를 걸어 계단 현상(앨리어싱)을 막는 처리.
- **고전 컴퓨터 비전:** Canny 에지 검출 (가우시안 블러 → Sobel), Harris 코너 검출 (파노라마 이어 붙이기), HOG와 SIFT 특징 (기울기 방향의 분포; HOG + SVM 보행자 검출은 2000년대의 대표 기법).
- **산업 현장:** 제품 표면의 긁힘 검사, 문서 스캔과 OCR 전처리, 고전적인 차선 인식. 불량 데이터가 적어 학습이 어렵거나, GPU 없는 임베디드 기기처럼 계산 자원이 부족할 때 특히 쓰인다.
- **딥러닝 안에서:** 데이터 증강 (무작위 블러, 선명하게), 흐린 사진 걸러내기 (Laplacian 출력의 분산이 작으면 흐린 사진), 이미지 생성·복원 모델의 에지 손실 (Sobel 출력끼리 비교), 풀링 앞에 고정된 블러 필터를 넣는 BlurPool.

고정된 필터는 **데이터가 필요 없고, 해석할 수 있고, 빠르다.** 대신 에지나 블러처럼 사람이 수식으로 정의할 수 있는 특징까지만 다룰 수 있다.

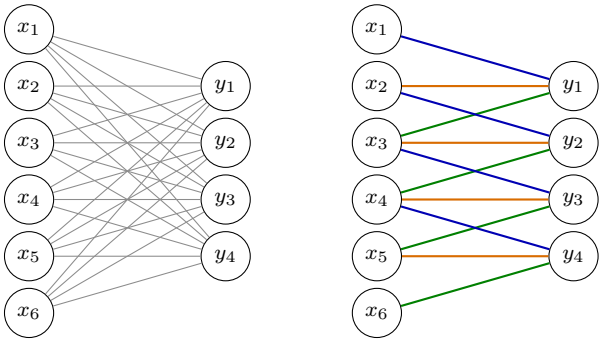
사람이 설계한 필터에서 학습하는 필터로

- 필터 연산은 **작은 무늬 검출기**이다. 출력은 각 위치에서 그 무늬가 얼마나 강하게 나타나는지를 담은 2차원 지도, 즉 **특징 맵**이다.
- 위의 필터들은 사람이 목적에 맞게 값을 정하였다. 그러나 복잡한 이미지를 분류해 내기 위해서 어떤 필터를 적용해야 할지, 어떻게 필터의 값은 정하기는 어렵다.
- CNN은 필터의 값을 **데이터로부터 학습**하는 **가중치로** 취급한다. 즉 필터가 학습 파라미터인 것이다. 학습된 첫 번째 층의 필터도 Sobel처럼 여러 방향의 에지에 반응하고 (6.1절), 층이 깊어질수록 더 복잡한 무늬를 찾는 필터가 된다 (6.2절).

필터가 가중치이다. 완전연결 신경망에는 가중치 W와 편향 B가 있었다. CNN에서는 **필터의 원소가 가중치**에 해당한다. 필터의 값을 학습하는 것이다. 편향은 필터를 적용한 뒤 모든 원소에 같은 값 하나를 더한다.



합성곱 계층은 연결을 제한한 완전연결 계층이다. 합성곱 연산을 노드와 화살표로 그리면 완전연결 계층과 비교할 수 있다. 간단히 1차원 입력 $x_1, \ldots, x_6$에 길이 3인 필터 (w_1, w_2, w_3)를 스트라이드 1로 적용하면 출력은 $y_j = w_1 x_j + w_2 x_{j+1} + w_3 x_{j+2}$ $(j = 1, \ldots, 4)$이다.



(a) 완전연결 가중치 $6 \times 4 = 24$개 (b) 합성곱 가중치 3개 (색이 같으면 같은 값) (c) (b)의 가중치 행렬 (6×4) 빈칸은 0

같은 입력과 출력에 대한 완전연결 계층과 합성곱 계층. $Y = XW$로 쓰면 합성곱은 W의 대부분이 0이고 같은 값이 대각선을 따라 반복되는 특별한 완전연결 계층이다.

그림에서 보듯이 합성곱 계층은 완전연결 계층에 두 가지 규칙을 더한 것이다.

- **국소 연결 (sparse connectivity):** 출력 노드 y_j는 모든 입력이 아니라 **공간적으로 인접한** 입력 x_j, x_{j+1}, x_{j+2}에만 연결된다. 이미지의 기하 구조 (가까운 픽셀끼리 관련이 깊다)를 연결 방식에 반영한 것이다. 이 인접한 영역을 y_j의 **수용 영역 (receptive field)**이라 한다.
- **가중치 공유 (parameter sharing):** 모든 출력 노드가 **같은 가중치** (w_1, w_2, w_3)를 쓴다. 같은 무늬 검출기를 모든 위치에 적용하는 것이므로, 무늬가 이미지의 어디에 있든 같은 방식으로 찾아낸다.

그래서 가중치 행렬 W로 쓰면 각 열에 w_1, w_2, w_3이 한 칸씩 내려가며 놓이고 나머지는 0인 띠 모양 행렬이 된다. 2차원 이미지에서도 마찬가지로, 출력 노드 하나는 입력의 $FH \times FW$ 영역에만 연결되고 모든 위치가 같은 필터를 공유한다. 예를 들어 MNIST (28×28)에 5×5 필터 하나를 적용하여 24×24 출력을 만들 때, 같은 입출력 크기의 완전연결 계층은 가중치가 $784 \times 576 = 451584$개이지만 합성곱 계층은 25개뿐이다. (이런 그림은 Goodfellow, Bengio, Courville의 *Deep Learning* 9장에서 sparse interactions와 parameter sharing을 설명할 때 쓰인다.)

코드로 확인하기. `conv_basics.py`에서 채널 하나, 필터 하나인 합성곱을 반복문으로 쓰면 다음과 같다.

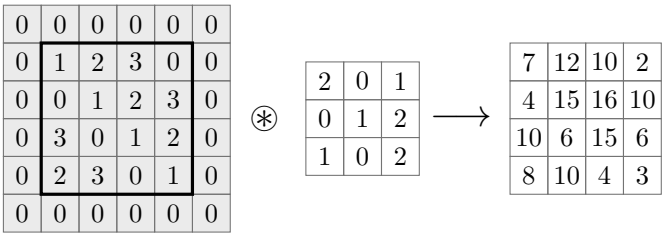
```
def conv2d_naive(x, f, stride=1, pad=0):
    """2-D convolution of one image x (H, W) with one filter f (FH, FW), with loops"""
    x = np.pad(x, pad)                                  # zeros around the image
    FH, FW = f.shape
    OH = (x.shape[0] - FH) // stride + 1
    OW = (x.shape[1] - FW) // stride + 1
    y = np.zeros((OH, OW))
    for i in range(OH):
        for j in range(OW):
            window = x[i * stride:i * stride + FH, j * stride:j * stride + FW]
            y[i, j] = np.sum(window * f)                # multiply elementwise, then
    add
    return y
```

```
print("convolution (4x4 input, 3x3 filter):\n", conv2d_naive(x, f))
print("with bias 3:\n", conv2d_naive(x, f) + 3)
print("padding 1:\n", conv2d_naive(x, f, pad=1))
```

```
convolution (4x4 input, 3x3 filter):
 [[15. 16.]
 [ 6. 15.]]
with bias 3:
 [[18. 19.]
 [ 9. 18.]]
```

2.3 패딩

합성곱 연산을 하기 전에 입력 데이터 주변을 특정 값(보통 0)으로 채우기도 한다. 이것을 **패딩(padding)**이라 한다. 아래는 $(4,4)$ 입력에 폭 1의 패딩을 적용한 예이다.



$(4,4)$ 입력 + 패딩 1 $(3,3)$ 필터 $(4,4)$ 출력

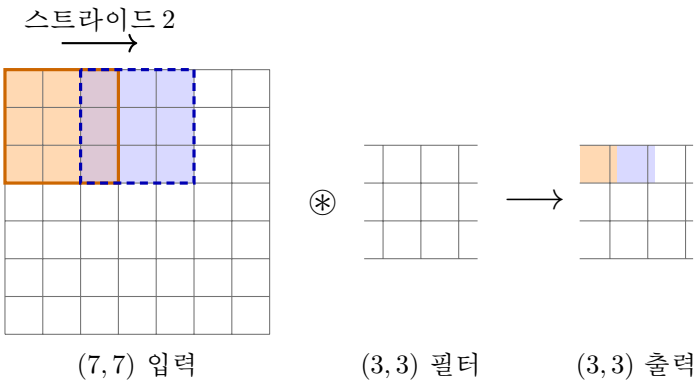
패딩이 없으면 $(4,4)$ 입력이 $(2,2)$로 줄어들지만, 패딩을 1 하면 출력도 $(4,4)$로 입력과 같은 크기가 된다.

```
padding 1:
 [[ 7. 12. 10.  2.]
 [ 4. 15. 16. 10.]
 [10.  6. 15.  6.]
 [ 8. 10.  4.  3.]]
```

패딩은 주로 **출력 크기를 조정**하려고 쓴다. 합성곱 연산을 할 때마다 크기가 줄어들면, 합성곱을 여러 번 거치는 깊은 network에서는 어느 시점에 출력이 1이 되어 더 이상 합성곱을 할 수 없게 된다. 패딩을 쓰면 공간 크기를 유지한 채 다음 계층에 전달할 수 있다. 또 패딩이 없으면 가장자리의 픽셀은 필터가 몇 번밖에 지나가지 않아 정보가 덜 반영된다.

2.4 스트라이드

필터를 적용하는 위치의 간격을 **스트라이드(stride)**라 한다. 지금까지는 스트라이드가 1이었다. 스트라이드를 2로 하면 윈도우가 두 칸씩 이동한다.



스트라이드를 키우면 출력이 작아지고, 패딩을 키우면 출력이 커진다. 입력 크기를 (H, W), 필터 크기를 (FH, FW), 패딩을 P, 스트라이드를 S라 하면 출력 크기 (OH, OW)는

합성곱의 출력 크기

$$OH = \left\lfloor \frac{H + 2P - FH}{S} \right\rfloor + 1, \qquad OW = \left\lfloor \frac{W + 2P - FW}{S} \right\rfloor + 1$$

이다. 예를 들어

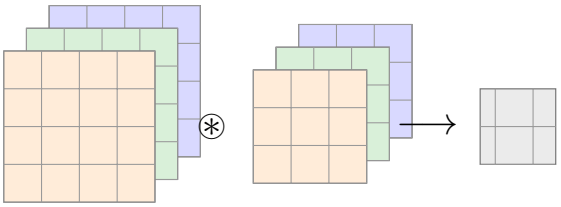
- 입력 $(4, 4)$, 패딩 1, 스트라이드 1, 필터 $(3, 3)$: $OH = \frac{4+2-3}{1} + 1 = 4$
- 입력 $(7, 7)$, 패딩 0, 스트라이드 2, 필터 $(3, 3)$: $OH = \frac{7+0-3}{2} + 1 = 3$
- 입력 $(28, 31)$, 패딩 2, 스트라이드 3, 필터 $(5, 5)$: $OH = \frac{28+4-5}{3} + 1 = 10, OW = \frac{31+4-5}{3} + 1 = 11$

$\lfloor \cdot \rfloor$는 내림(floor)이다. 나누어떨어지지 않으면 남는 부분에는 필터를 놓지 않는다 (예: $H = 8$, $FH = 3$, $S = 2$, $P = 0$이면 $\lfloor 5/2 \rfloor + 1 = 3$). 코드의 `//`와 PyTorch도 같다. 위의 예처럼 나누어떨어지면 내림 기호를 생략해도 된다.

```
7x7 input, 3x3 filter, stride 1 -> shape (5, 5)
7x7 input, 3x3 filter, stride 2 -> shape (3, 3)
```

2.5 3차원 데이터의 합성곱

이미지는 보통 세로, 가로에 **채널** 방향까지 있는 3차원 데이터이다 (컬러 이미지는 RGB의 3채널). 채널이 여러 개이면 **채널마다 입력과 필터의 합성곱**을 하고, 그 결과를 **모두 더해서** 하나의 출력을 얻는다.



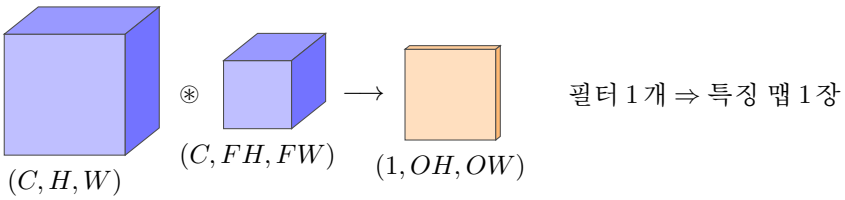
3차원 합성곱의 규칙

입력 데이터의 채널 수와 필터의 채널 수가 같아야 한다. 필터의 크기 (FH, FW)는 원하는 값으로 정할 수 있지만, 모든 채널의 필터는 같은 크기여야 한다. 채널 C개인 입력에 필터 하나를 적용하면 출력은 **채널 1개**의 특징 맵이 된다.

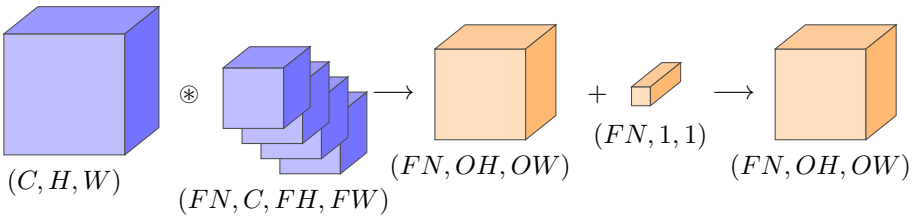
$$y_{ij} = \sum_{c=1}^{C} \sum_{p} \sum_{q} x_{c,\, i+p,\, j+q}\, f_{c,\, p,\, q} + b$$

2.6 블록으로 생각하기

3차원 합성곱은 데이터와 필터를 직육면체 **블록**으로 생각하면 이해하기 쉽다. 3차원 데이터의 shape을 (채널, 높이, 너비) $= (C, H, W)$ 순서로 쓴다. 필터는 (C, FH, FW)이다.



출력이 한 장(채널 1개)이면 다음 계층에 넘길 정보가 너무 적다. **필터를 FN개** 쓰면 출력 특징 맵도 FN장이 되어 (FN, OH, OW) 블록이 된다. 따라서 합성곱 계층의 필터(가중치)는 **4차원** (FN, C, FH, FW)이고, 편향은 필터마다 하나씩 FN개이다 (shape $(FN, 1, 1)$로 보면 broadcast로 더해진다).



필터 FN개를 쓰는 합성곱 계층. 편향은 필터(출력 채널)마다 하나이다.

왜 합성곱인가: 가중치 공유

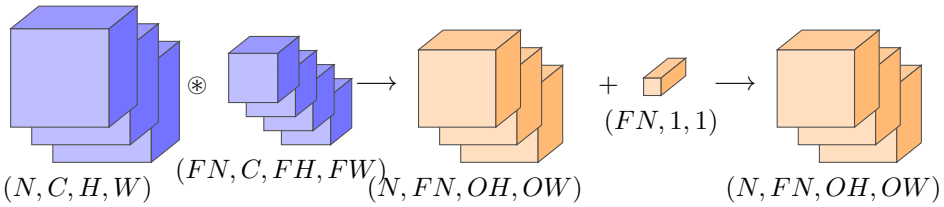
합성곱 계층에서는 **같은 필터를 이미지의 모든 위치에 적용**한다 (가중치 공유, weight sharing). 그래서 세로 획을 찾는 필터 하나가 이미지의 왼쪽 위에 있는 세로 획과 오른쪽 아래에 있는 세로 획을 똑같이 찾아낸다. 또 매개변수가 아주 적다. MNIST 이미지 $(1, 28, 28)$에서 $(30, 24, 24)$의 출력을 만든다고 하자.

- 완전연결 계층: $784 \times (30 \cdot 24 \cdot 24) = 784 \times 17280 \approx 1350$만 개의 가중치
- 합성곱 계층 (5×5 필터 30개): $30 \times 1 \times 5 \times 5 = 750$개의 가중치와 30개의 편향

각 출력 원소는 입력의 5×5 영역만 본다 (**국소 연결**). 이 두 성질이 이미지의 구조에 잘 맞는다.

2.7 배치 처리

완전연결 계층에서 미니배치로 데이터를 묶어 한 번에 처리했듯이, 합성곱 계층도 배치 처리를 한다. 각 계층을 흐르는 데이터에 차원을 하나 더 늘려 **4차원** (데이터 수, 채널, 높이, 너비) $= (N, C, H, W)$로 저장한다. (지금까지 미니배치 크기를 h라 했지만, 이 장에서는 높이 H와 헷갈리지 않도록 N이라 쓴다.)

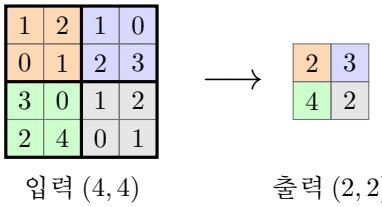


합성곱 연산의 배치 처리. N개의 데이터에 대한 합성곱을 한 번에 한다.

신경망에는 4차원 데이터가 하나 흐를 때마다 데이터 N개에 대한 합성곱 연산이 이루어진다.

3 풀링 계층

풀링(pooling)은 세로, 가로 방향의 공간을 줄이는 연산이다. 아래는 2×2 **최대 풀링(max pooling)**을 스트라이드 2로 하는 예이다. 2×2 영역마다 가장 큰 원소 하나를 꺼낸다.



풀링의 윈도우 크기와 스트라이드는 보통 같은 값으로 한다 (윈도우 3×3이면 스트라이드 3). 최대 풀링 외에 영역의 평균을 내는 **평균 풀링(average pooling)**도 있지만, 이미지 인식에서는 주로 최대 풀링을 쓴다. 이 노트에서 풀링이라 하면 최대 풀링을 뜻한다.

풀링 계층의 특징

- **학습해야 할 매개변수가 없다.** 영역에서 최댓값(또는 평균)을 취하는 명확한 처리일 뿐이다.
- **채널 수가 변하지 않는다.** 채널마다 독립적으로 계산한다: $(N, C, H, W) \rightarrow (N, C, OH, OW)$.
- **입력의 작은 변화에 영향을 적게 받는다(robust).** 입력이 한두 칸 이동해도 영역 안의 최댓값은 그대로인 경우가 많다.

```
pool = Pooling(2, 2)                    # max pooling, 2x2 window, stride 2
```

```
2x2 max pooling:
 [[2. 3.]
 [4. 2.]]
```

4 합성곱 / 풀링 계층 구현하기

지금까지 그래 왔듯이 합성곱 계층과 풀링 계층도 `forward`와 `backward`를 가진 클래스로 구현한다. 코드는 `code/conv_layers.py`에 있고, Affine, ReLU, Softmax 계층은 지난 시간의 `layers.py`를 그대로 쓴다.

파일	내용
`conv_layers.py`	`im2col`, `col2im`, `Convolution`, `Pooling`, `Flatten`, `SimpleConvNet`
`layers.py`, `optimizers.py`, `mnist_data.py`	지난 시간의 모듈 (그대로 복사)
`cat_parts.py`	도입부의 고양이 사진과 부분 그림
`classic_filters.py`	사람이 설계한 필터 (평균, Sobel, Laplacian 등)를 흑백 사진에 적용
`conv_basics.py`	이 노트의 예제, im2col 확인과 속도 비교, gradient 확인
`mnist_cnn.py`	SimpleConvNet으로 MNIST 학습, 필터와 특징 맵 시각화
`mnist_cnn_torch.py`	같은 network를 PyTorch로

4.1 4차원 배열

CNN에서 계층 사이를 흐르는 데이터는 4차원 배열 (N, C, H, W)이다. 예를 들어 높이 28, 너비 28, 채널 1인 데이터가 10개라면 shape은 $(10, 1, 28, 28)$이다.

```
>>> x = np.random.rand(10, 1, 28, 28)
>>> x[0].shape         # the first image: (1, 28, 28)
>>> x[0, 0].shape      # its first channel: (28, 28)
```

지금까지 MNIST를 $(N, 784)$로 다루었지만, CNN에서는 `reshape`으로 모양을 되살려 입력한다 (`mnist_cnn.py`).

```
x_train, t_train, x_val, t_val, x_test, t_test = load_mnist_split()
# a CNN keeps the SHAPE of the image: (N, 784) -> (N, 1, 28, 28) = (images, channels,
    height, width)
x_train = x_train.reshape(-1, 1, 28, 28)
```

4.2 im2col로 데이터 전개하기

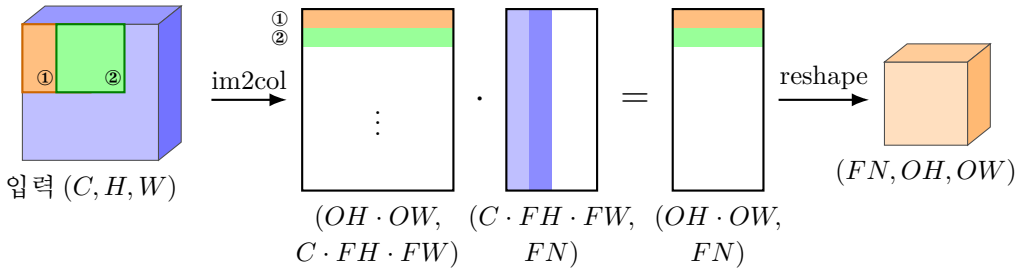
합성곱을 2.2절의 `conv2d_naive`처럼 반복문으로 구현하면, 데이터 N개, 필터 FN개, 출력의 모든 위치 $OH \times OW$에 대하여 `for` 문이 겹겹이 필요하다. numpy에서 `for` 문은 느리다. 그래서

im2col(image to column)이라는 함수로 입력 데이터를 필터를 적용하기 좋게 **2차원 행렬로 펼친다.**

im2col의 아이디어

필터가 방문하는 위치 하나마다, 그 위치의 입력 영역 (C, FH, FW)을 **한 행**으로 펼친다.

- 입력 $(N, C, H, W) \xrightarrow{\text{im2col}}$ 행렬 $(N \cdot OH \cdot OW,\ C \cdot FH \cdot FW)$
- 필터 $(FN, C, FH, FW) \xrightarrow{\text{reshape}}$ 필터 하나를 **한 열**로: $(C \cdot FH \cdot FW,\ FN)$
- 합성곱 = 두 행렬의 곱 + 편향: $(N \cdot OH \cdot OW,\ FN) \xrightarrow{\text{reshape}} (N, FN, OH, OW)$



im2col을 이용한 합성곱 (데이터 하나의 경우). 필터의 위치 ①, ②, ... 이 행렬의 각 행이 된다.

이렇게 하면 합성곱 계층의 순전파는 **Affine 계층과 똑같은** 행렬곱 $A = \text{col} \cdot W_{\text{col}} + B$가 된다. 행렬곱은 numpy가 최적화된 선형대수 라이브러리(BLAS)로 계산하므로 매우 빠르다.

```python
def im2col(X, FH, FW, stride=1, pad=0):
    """(N, C, H, W) -> (N*OH*OW, C*FH*FW): ONE ROW for every place the filter visits"""
    N, C, H, W = X.shape
    OH, OW = output_size(H, FH, stride, pad), output_size(W, FW, stride, pad)
    Xp = np.pad(X, [(0, 0), (0, 0), (pad, pad), (pad, pad)])    # zero padding
    col = np.zeros((N, C, FH, FW, OH, OW))
    for i in range(FH):                    # loop over the FILTER, not the image
        for j in range(FW):
            col[:, :, i, j, :, :] = Xp[:, :, i:i + stride * OH:stride, j:j + stride *
    OW:stride]
    return col.transpose(0, 4, 5, 1, 2, 3).reshape(N * OH * OW, -1)
```

반복문은 이미지 전체가 아니라 **필터의 크기** ($FH \times FW$, 예: $5 \times 5 = 25$번)만큼만 돈다. `(i, j)` 번째 반복에서는 "필터의 (i, j) 원소와 곱해질 입력의 값들"을 모든 출력 위치에 대해 한꺼번에 잘라온다. 마지막 `transpose`와 `reshape`으로 출력 위치마다 한 행이 되도록 정리한다.

```python
x1 = np.random.rand(1, 3, 7, 7)                    # 1 image, 3 channels, 7x7
print("im2col of (1, 3, 7, 7) with a 5x5 filter:", im2col(x1, 5, 5).shape)
x2 = np.random.rand(10, 3, 7, 7)                   # 10 images
print("im2col of (10, 3, 7, 7) with a 5x5 filter:", im2col(x2, 5, 5).shape)
```

```
im2col of (1, 3, 7, 7) with a 5x5 filter: (9, 75)
im2col of (10, 3, 7, 7) with a 5x5 filter: (90, 75)
```

7×7 입력에 5×5 필터를 적용하면 출력은 $3 \times 3 = 9$개의 위치이고, 각 위치의 입력 영역은 $3 \times 5 \times 5 = 75$개의 값이다. 데이터가 10개이면 행이 90개가 된다.

얼마나 빠를까? MNIST 크기의 미니배치 $(100, 1, 28, 28)$에 5×5 필터 30개를 적용하는 합성곱을, 네 겹의 반복문과 im2col로 각각 계산해 비교한다.

```
def conv_loops(X, W, B):
    """(N, C, H, W) with FN filters (FN, C, FH, FW): every image, every filter, every
    position"""
    N, C, H, Wd = X.shape
    FN, _, FH, FW = W.shape
    OH, OW = H - FH + 1, Wd - FW + 1
    Y = np.zeros((N, FN, OH, OW))
    for n in range(N):
        for k in range(FN):
            for i in range(OH):
                for j in range(OW):
                    Y[n, k, i, j] = np.sum(X[n, :, i:i + FH, j:j + FW] * W[k]) + B[k]
    return Y
```

```
output shape (100, 30, 24, 24), max |difference| = 5.3e-15
loops 5.74 s,  im2col 0.057 s  (101 times faster)
```

결과는 같고 (차이 10^{-15}), im2col이 약 100배 빠르다. 반복문으로는 미니배치 하나에 6초가 걸리므로, 550개의 미니배치로 이루어진 한 epoch에 거의 1시간이 걸린다.

im2col의 대가: 메모리

필터가 겹쳐서 이동하면 입력의 같은 값이 im2col 행렬의 여러 행에 중복해서 들어간다. $(100, 1, 28, 28)$ 입력 (78400개의 값)이 5×5 필터로 전개되면 $(100 \cdot 24 \cdot 24,\ 25) = (57600, 25)$, 즉 144만 개의 값이 된다. 약 18배이다. 메모리를 더 쓰는 대신 속도를 얻는 것이다. 그래서 `SimpleConvNet`의 `accuracy`는 10000장을 한꺼번에 넣지 않고 1000장씩 나누어 계산한다. GPU용 라이브러리(cuDNN 등)는 im2col 외에도 FFT, Winograd 같은 여러 방법을 상황에 따라 골라 쓴다.

4.3 합성곱 계층 구현하기

im2col을 쓰면 합성곱 계층의 순전파는 "im2col → Affine → reshape"이다.

```
class Convolution:
    """Convolution of (N, C, H, W) with FN filters (FN, C, FH, FW) -> (N, FN, OH, OW).
       With im2col: ONE matrix product, i.e. an Affine layer at every filter position."
    ""
    def __init__(self, W, B, stride=1, pad=0):
        self.W = W                  # (FN, C, FH, FW)
        self.B = B                  # (FN,): one bias per filter
        self.stride = stride
        self.pad = pad
        self.dW = None
        self.dB = None

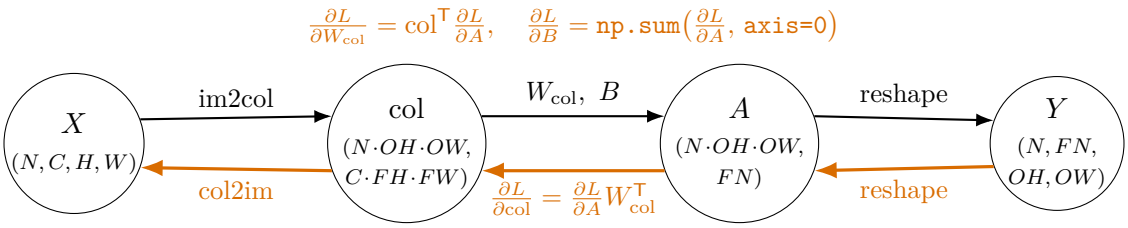
    def forward(self, X):
        FN, C, FH, FW = self.W.shape
        N, _, H, W = X.shape
```

```
        OH = output_size(H, FH, self.stride, self.pad)
        OW = output_size(W, FW, self.stride, self.pad)
        self.X_shape = X.shape
        self.col = im2col(X, FH, FW, self.stride, self.pad)      # (N*OH*OW, C*FH*FW)
        self.W_col = self.W.reshape(FN, -1).T                     # one filter per
    COLUMN
        A = self.col @ self.W_col + self.B                        # (N*OH*OW, FN)
        return A.reshape(N, OH, OW, FN).transpose(0, 3, 1, 2)     # (N, FN, OH, OW)
```

A의 각 행은 (데이터, 출력 위치) 하나, 각 열은 필터(출력 채널) 하나이다. 그래서 `reshape(N, OH, OW, FN)`으로 나눈 뒤, 채널 축을 두 번째로 옮겨 (N, FN, OH, OW)를 만든다. 출력 크기는 2.4절의 식을 그대로 함수로 만든 것이다.

```
def output_size(H, FH, stride=1, pad=0):
    """height (or width) of the output: OH = (H + 2P - FH) / S + 1"""
    return (H + 2 * pad - FH) // stride + 1
```

합성곱 계층의 역전파. 순전파가 세 단계로 나뉘었으므로, 변수 노드 기준의 계산 그래프로 그리면 다음과 같다.



합성곱 계층의 계산 그래프. 가운데는 지난 시간의 Affine 계층 그대로이다.

- **reshape의 역전파**는 gradient의 모양을 원래대로 되돌리는 것뿐이다 (값은 그대로 전달).
- **가운데는 Affine 계층**이다. 지난 시간에 유도한 $\frac{\partial L}{\partial W} = X^\top \frac{\partial L}{\partial A}$, $\frac{\partial L}{\partial B} = \sum_{\text{행}} \frac{\partial L}{\partial A}$, $\frac{\partial L}{\partial X} = \frac{\partial L}{\partial A} W^\top$를 그대로 쓴다.
- **im2col의 역전파는 col2im**이다. im2col에서 입력의 한 값이 col의 여러 행에 복사되었으므로 (필터 위치가 겹치는 곳), 역전파에서는 그 값으로 돌아오는 gradient를 **모두 더해야** 한다. 한 변수에서 화살표가 여러 개 나가면 gradient를 더한다는 다변수 연쇄법칙(배치 정규화에서 보았다)과 같은 원리이다.

```
    def backward(self, dA):
        FN, C, FH, FW = self.W.shape
        dA = dA.transpose(0, 2, 3, 1).reshape(-1, FN)             # (N*OH*OW, FN)
        self.dB = np.sum(dA, axis=0)                              # the Affine rules
        self.dW = (self.col.T @ dA).T.reshape(FN, C, FH, FW)
        dcol = dA @ self.W_col.T                                  # (N*OH*OW, C*FH*FW)
        return col2im(dcol, self.X_shape, FH, FW, self.stride, self.pad)
```

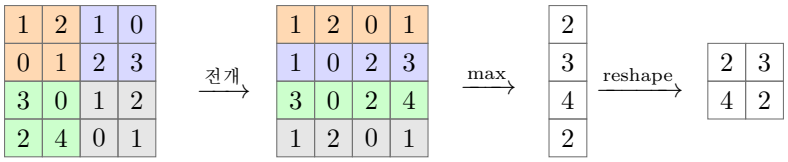
```
def col2im(col, X_shape, FH, FW, stride=1, pad=0):
    """the reverse of im2col: ADD every row back where it came from"""
```

```
    N, C, H, W = X_shape
    OH, OW = output_size(H, FH, stride, pad), output_size(W, FW, stride, pad)
    col = col.reshape(N, OH, OW, C, FH, FW).transpose(0, 3, 4, 5, 1, 2)
    Xp = np.zeros((N, C, H + 2 * pad, W + 2 * pad))
    for i in range(FH):
        for j in range(FW):
            Xp[:, :, i:i + stride * OH:stride, j:j + stride * OW:stride] += col[:, :, i
    , j, :, :]
    return Xp[:, :, pad:pad + H, pad:pad + W]      # remove the padding
```

im2col에서 =로 잘라 온 자리에, col2im에서는 +=로 더해 넣는다는 것만 다르다.

4.4 풀링 계층 구현하기

풀링 계층도 im2col로 입력을 전개한다. 다만 풀링은 **채널마다 독립적**이므로, 전개한 뒤 한 행이 "한 채널의 한 윈도우"가 되도록 다시 reshape하고, 행마다 최댓값을 구한다.



풀링 계층의 구현: 윈도우마다 한 행으로 전개하고, 행마다 최댓값을 구한 뒤 모양을 되돌린다.

```
    def forward(self, X):
        N, C, H, W = X.shape
        OH = output_size(H, self.PH, self.stride, self.pad)
        OW = output_size(W, self.PW, self.stride, self.pad)
        col = im2col(X, self.PH, self.PW, self.stride, self.pad)
        col = col.reshape(-1, self.PH * self.PW)           # one window per row
        self.argmax = np.argmax(col, axis=1)               # where the maximum was
        self.X_shape = X.shape
        Y = np.max(col, axis=1)
        return Y.reshape(N, OH, OW, C).transpose(0, 3, 1, 2)   # (N, C, OH, OW)
```

풀링 계층의 역전파. $y = \max(x_1, x_2, x_3, x_4)$에서 최댓값이 x_k이면 $\frac{\partial y}{\partial x_k} = 1$이고 나머지는 0이다. 즉 최댓값이었던 원소에만 gradient를 그대로 전달하고, 나머지에는 0을 보낸다. ReLU에서 mask로 통과한 원소에만 gradient를 보낸 것과 같다. 순전파에서 최댓값의 위치(argmax)를 기억해 두는 이유이다. (최댓값이 여러 개로 같으면 미분이 하나로 정해지지 않는다. 이 구현은 np.argmax가 고른 첫 위치로 보낸다.)

```
    def backward(self, dY):
        dY = dY.transpose(0, 2, 3, 1).ravel()               # the same order as col
        dcol = np.zeros((dY.size, self.PH * self.PW))
        dcol[np.arange(dY.size), self.argmax] = dY          # only the max gets it
        dcol = dcol.reshape(-1, self.X_shape[1] * self.PH * self.PW)
        return col2im(dcol, self.X_shape, self.PH, self.PW, self.stride, self.pad)
```

4.5 Flatten 계층

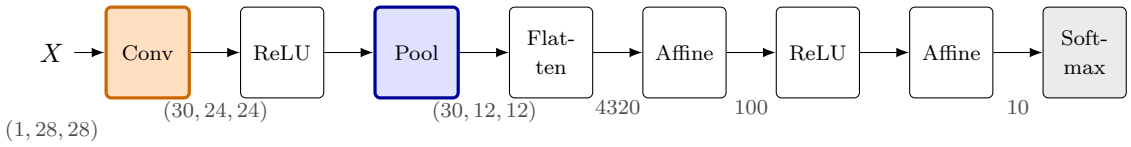
합성곱과 풀링을 거친 4차원 데이터를 Affine 계층에 넣으려면 데이터마다 한 줄로 펼쳐야 한다. 이것을 계층 하나로 만들어 둔다 (PyTorch의 `nn.Flatten`). 역전파는 모양을 되돌리는 것뿐이다.

```
class Flatten:
    """(N, C, H, W) -> (N, C*H*W) before an Affine layer (nn.Flatten in PyTorch)"""
    def forward(self, X):
        self.X_shape = X.shape
        return X.reshape(X.shape[0], -1)

    def backward(self, dY):
        return dY.reshape(self.X_shape)
```

5 CNN 구현하기

합성곱 계층과 풀링 계층을 조합하여 손글씨 숫자를 인식하는 CNN (`SimpleConvNet`)을 만든다.



SimpleConvNet의 구조와 각 계층을 지난 뒤 데이터 하나의 shape

계층	설정	출력 shape	매개변수 수
입력		$(1, 28, 28)$	
Conv	5×5 필터 30개	$(30, 24, 24)$	$30 \cdot 1 \cdot 5 \cdot 5 + 30 = 780$
ReLU		$(30, 24, 24)$	
Pooling	2×2 최대 풀링	$(30, 12, 12)$	
Flatten		4320	
Affine		100	$4320 \cdot 100 + 100 = 432100$
ReLU		100	
Affine		10	$100 \cdot 10 + 10 = 1010$
합계			433890

(합성곱은 스트라이드 1, 패딩 0, 풀링은 스트라이드 2이다. 출력 shape은 데이터 하나의 것이다.)

합성곱 계층의 매개변수는 780개뿐이고, 매개변수의 99.6%는 첫 번째 Affine 계층에 있다. 합성곱 계층이 가중치를 공유하기 때문이다 (2.6절).

`SimpleConvNet`은 지난 시간의 `MultiLayerNet`과 같은 방식으로 만든다. 매개변수를 `params` 딕셔너리에 두고, 같은 배열을 계층에도 넘겨주어 optimizer가 `params`를 갱신하면 계층의 가중치도 바뀌게 한다. 초깃값의 표준편차는 `weight_init_std`로 정한다. `"he"`이면 He 초깃값이고 (합성곱 필터 하나가 보는 입력의 수는 $C \cdot FH \cdot FW = 25$개이므로 표준편차 $\sqrt{2/25}$), 숫자이면 그 값을 표준편차로 쓴다.

```
class SimpleConvNet:
    """Conv -> ReLU -> Pool -> Flatten -> Affine -> ReLU -> Affine -> Softmax"""
```

```
    def __init__(self, input_shape=(1, 28, 28), filter_num=30, filter_size=5,
                 hidden_size=100, output_size=10, weight_init_std="he"):
        C, H, W = input_shape
        conv_out = H - filter_size + 1                      # 28 - 5 + 1 = 24
        pool_out = conv_out // 2                            # 2x2 max pooling: 12
        flat = filter_num * pool_out * pool_out             # 30 * 12 * 12 = 4320
        n_in = C * filter_size * filter_size                # 25 inputs per filter
        def std(n):                                         # "he": sqrt(2 / n_in), or a
     fixed number such as 0.01
            return np.sqrt(2.0 / n) if weight_init_std == "he" else weight_init_std
        self.params = {
            "W1": std(n_in) * np.random.randn(filter_num, C, filter_size, filter_size),
            "B1": np.zeros(filter_num),
            "W2": std(flat) * np.random.randn(flat, hidden_size),
            "B2": np.zeros((1, hidden_size)),
            "W3": std(hidden_size) * np.random.randn(hidden_size, output_size),
            "B3": np.zeros((1, output_size)),
        }
        self.layers = [
            Convolution(self.params["W1"], self.params["B1"]),
            Relu(),
            Pooling(2, 2),
            Flatten(),
            Affine(self.params["W2"], self.params["B2"]),
            Relu(),
            Affine(self.params["W3"], self.params["B3"]),
        ]
        self.last = SoftmaxWithCrossEntropy()
```

순전파, 손실, 역전파는 MultiLayerNet과 똑같다. 계층이 모두 같은 forward, backward 형식을 따르기 때문에, 새로운 계층이 들어와도 network 코드는 바뀌지 않는다. 이것이 계층 단위로 구현하는 장점이다.

```
    def backward(self):                         # call right after loss()
        d = self.last.backward()
        for layer in reversed(self.layers):     # layer after layer
            d = layer.backward(d)
        grads, k = {}, 0
        for layer in self.layers:
            if isinstance(layer, (Convolution, Affine)):
                k += 1
                grads[f"W{k}"], grads[f"B{k}"] = layer.dW, layer.dB
        return grads
```

gradient 확인. 작은 SimpleConvNet (입력 $(2, 8, 8)$, 3×3 필터 3개)으로 역전파와 수치미분을 비교한다 (conv_basics.py의 (3)).

```
np.random.seed(1)
small = SimpleConvNet(input_shape=(2, 8, 8), filter_num=3, filter_size=3, hidden_size
    =5, output_size=4)
X_small = np.random.randn(5, 2, 8, 8)                   # 5 images, 2 channels, 8x8
```

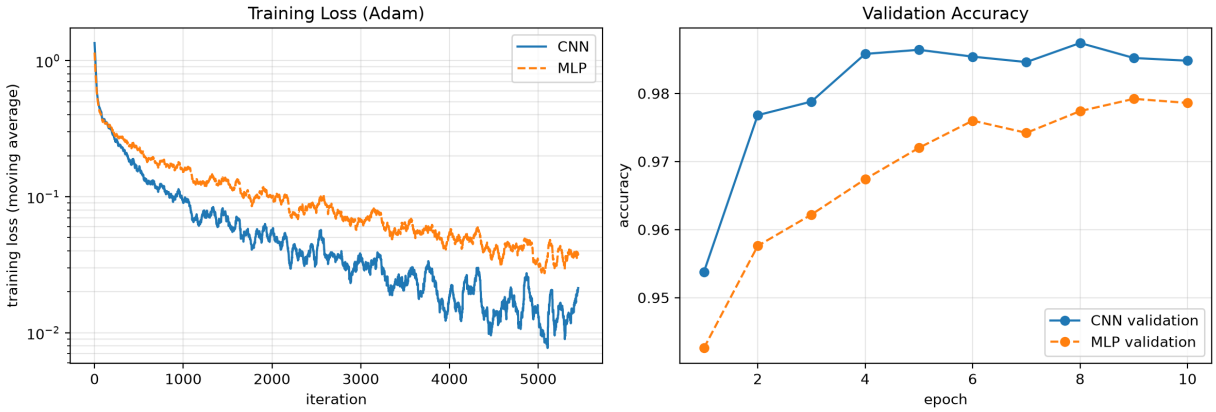
```
T_small = np.eye(4)[np.random.randint(0, 4, size=5)]
small.loss(X_small, T_small)
grads = small.backward()
```

```
W1: shape (3, 2, 3, 3)    max |backprop - numerical| = 1.15e-09
B1: shape (3,)            max |backprop - numerical| = 1.22e-09
W2: shape (27, 5)         max |backprop - numerical| = 1.42e-09
B2: shape (1, 5)          max |backprop - numerical| = 6.68e-10
W3: shape (5, 4)          max |backprop - numerical| = 1.03e-09
B3: shape (1, 4)          max |backprop - numerical| = 3.82e-10
```

합성곱, 풀링, Flatten을 지나는 역전파가 모두 올바르다.

MNIST로 학습하기. `mnist_cnn.py`에서 `SimpleConvNet`을 Adam (학습률 0.001, 미니배치 100)으로 10 epoch 학습시킨다. 초깃값은 표준편차 0.01의 작은 값으로 시작한다. 그래야 학습으로 생긴 필터의 무늬가 잘 드러난다 (6.1절). 비교를 위해 지난 시간의 다층 퍼셉트론(MLP, 784 → 100 → 10, He 초깃값)도 같은 조건으로 학습시킨다.

```
np.random.seed(1)
cnn = SimpleConvNet(input_shape=(1, 28, 28), filter_num=30, filter_size=5,
                    hidden_size=100, output_size=10,
                    weight_init_std=0.01)              # start small
print("SimpleConvNet parameters:", cnn.num_params())
W1_before = cnn.params["W1"].copy()                    # the filters before training
cnn_result = train(cnn)
```



SimpleConvNet과 MLP의 훈련 손실 (50 iteration 이동평균, 로그 눈금, 왼쪽)과 epoch별 검증 정확도 (오른쪽)

	검증 정확도						
	1 epoch	2 epoch	5 epoch	10 epoch	시험	매개변수	epoch당
SimpleConvNet	95.4%	97.7%	98.6%	98.5%	**98.5%**	433890	약 60초
MLP (784 → 100 → 10)	94.3%	95.8%	97.2%	97.9%	97.6%	79510	약 1초

- CNN은 2 **epoch 만에** 검증 정확도 97.7%로, MLP가 10 epoch 동안 도달한 97.9%에 거의 이르고, 4 epoch부터는 98.5% 이상을 유지한다. 시험 정확도는 98.5%로 MLP보다 약 1%p 높다. 틀리는 수로 보면 10000장 중 237장에서 154장으로 약 35% 줄었다.

- 손실도 CNN이 훨씬 빨리, 훨씬 낮게 내려간다.
- 다만 매개변수 수가 다르므로 공정한 비교는 아니다. CNN의 매개변수 대부분은 합성곱 계층이 아니라 Affine 계층에 있다. 합성곱 계층의 780개 매개변수가 이미지의 모양을 이용해 **좋은 특징**을 만들어 주기 때문에 뒤의 Affine 계층이 분류를 더 쉽게 하는 것이다.
- numpy로 구현한 CNN은 한 epoch에 약 1분이 걸린다. 합성곱은 계산량이 많아 GPU와 최적화된 라이브러리가 중요하다.

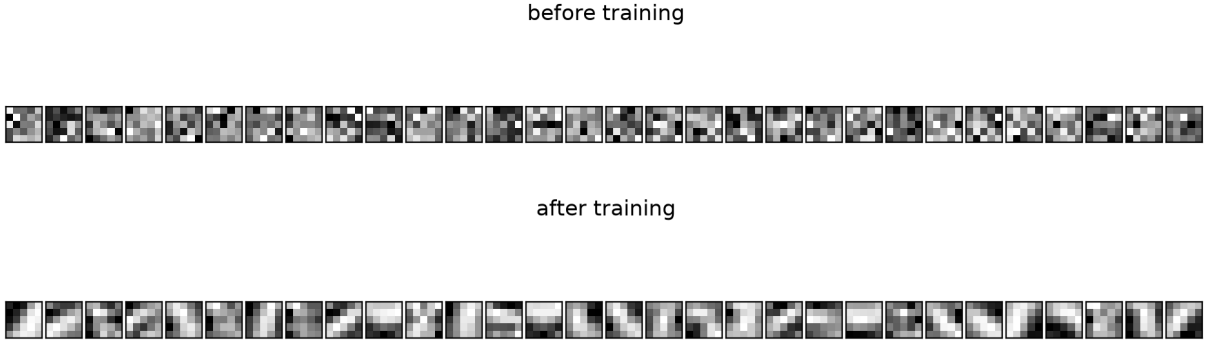
20 epoch 정도 학습하면 시험 정확도가 약 99%까지 올라간다. 이 노트에서는 시간을 줄이려고 10 epoch만 학습하였다.

6 CNN 시각화하기

CNN의 합성곱 계층은 무엇을 보고 있을까? 학습된 필터와 그 출력을 그려 본다.

6.1 첫 번째 층의 가중치 시각화하기

첫 번째 합성곱 계층의 필터는 $(30, 1, 5, 5)$, 즉 5×5 크기의 흑백 이미지 30장으로 볼 수 있다. 학습 전과 후의 필터를 이미지로 그리면 다음과 같다 (밝을수록 큰 가중치).

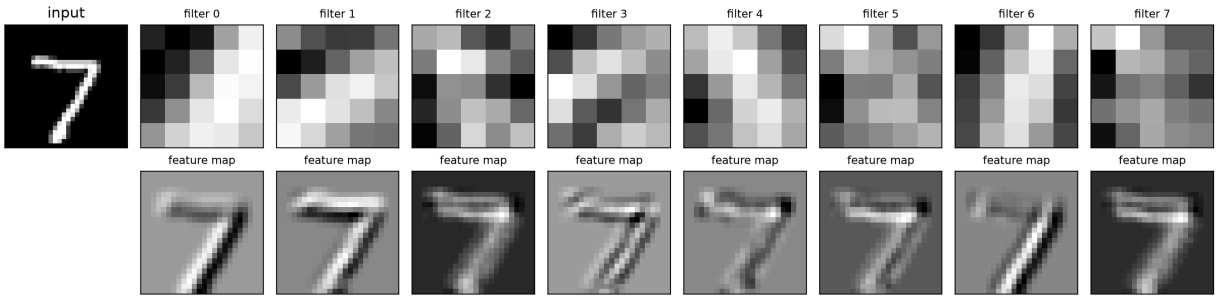


첫 번째 합성곱 계층의 필터 30개: 학습 전 (위)과 학습 후 (아래)

학습 전의 필터는 흑백의 정도가 무작위이지만, 학습 후의 필터는 흰색과 검은색이 **규칙적으로 나뉜** 모양으로 바뀌었다. 어떤 필터는 왼쪽이 어둡고 오른쪽이 밝으며 (세로 경계), 어떤 필터는 위가 어둡고 아래가 밝고 (가로 경계), 어떤 필터는 대각선 방향으로 밝은 띠가 있다. 흰색에서 검은색으로 바뀌는 경계, 즉 **에지**에 반응하는 필터가 학습된 것이다. 2절에서 본 Sobel 필터처럼 사람이 설계했던 필터를 network가 데이터로부터 스스로 찾아냈다.

필터는 무엇에 반응하는가: 특징 맵. 필터 자체보다 필터를 이미지에 적용한 결과, 즉 **특징 맵**을 보면 필터의 역할이 더 잘 드러난다. 학습된 network의 합성곱 계층에 시험 이미지 하나를 넣어 출력 $(1, 30, 24, 24)$ 중 앞의 8장을 그린다.

```
image = x_test[:1]                                  # (1, 1, 28, 28)
feature = cnn.layers[0].forward(image)              # (1, 30, 24, 24)
```

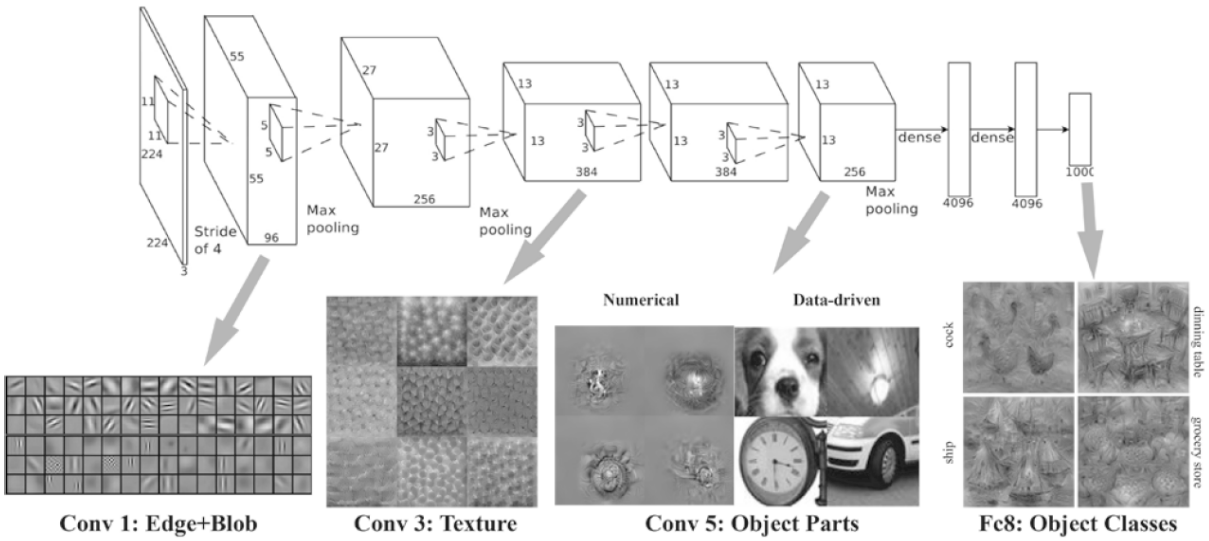


입력 이미지와 학습된 필터 8개 (위), 각 필터의 출력 특징 맵 (아래, ReLU 적용 전)

- 특징 맵마다 숫자 7의 **서로 다른 부분이 밝게**(큰 값) 나타난다. 예를 들어 필터 0 (왼쪽이 어둡고 오른쪽이 밝다)은 대각선 획의 왼쪽 가장자리에서 밝고 오른쪽 가장자리에서 어둡다. 필터 1 (위가 어둡고 아래가 밝다)은 위쪽 가로 획의 아래 가장자리에 강하게 반응하고, 필터 6 (가운데 세로로 밝은 띠)은 대각선 획 자체를 밝게 나타낸다.
- 즉 첫 번째 층의 필터는 특정 방향의 **에지**(색이 바뀌는 경계)에 반응한다. 같은 필터를 모든 위치에 적용하므로, 그 방향의 에지가 이미지의 어디에 있든 찾아낸다.
- 이렇게 만든 30장의 특징 맵을 풀링으로 줄인 뒤 Affine 계층이 조합하여 숫자를 판단한다.

6.2 층 깊이에 따른 추출 정보 변화

첫 번째 층의 필터는 에지나 블롭(blob, 국소적으로 덩어리진 영역) 같은 **저수준 정보**에 반응한다. 그렇다면 층을 여러 개 쌓으면 어떻게 될까? 대규모 이미지 데이터(ImageNet)로 학습한 8층 CNN (AlexNet, 7.2절)에서 각 층의 뉴런이 어떤 이미지에 가장 강하게 반응하는지 시각화한 결과는 다음과 같다 (Zeiler와 Fergus, 2014 등).

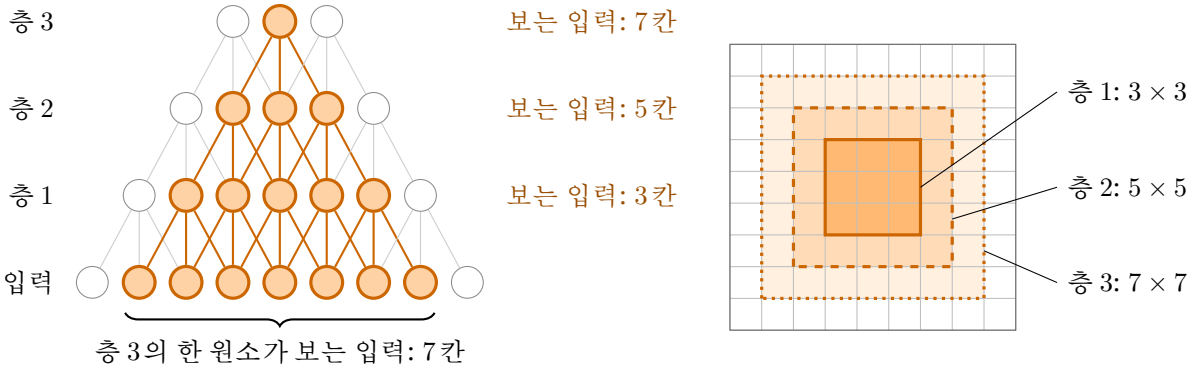


층 깊이에 따른 뉴런의 반응 대상. 위는 network의 구조 (합성곱 계층 5개 + 완전연결 계층 3개), 아래는 각 층의 뉴런이 강하게 반응하는 이미지이다.

- **Conv 1 (에지와 블롭):** 첫 번째 층의 필터는 여러 방향의 줄무늬와 색 덩어리 모양이다. 앞에서 본 Sobel 같은 에지 필터와 닮았다.

- **Conv 3 (텍스처):** 세 번째 층은 에지가 반복된 **무늬**(벌집, 비늘 같은 질감)에 반응한다.
- **Conv 5 (사물의 일부):** 다섯 번째 층은 개의 얼굴, 시계의 문자판, 자동차 바퀴처럼 **사물의 부분**에 반응한다. 왼쪽(Numerical)은 그 뉴런의 출력이 가장 커지도록 입력 이미지를 계산으로 만든 것이고, 오른쪽(Data-driven)은 실제 데이터 중에서 그 뉴런이 가장 강하게 반응한 이미지이다.
- **Fc8 (사물의 클래스):** 마지막 완전연결 계층의 뉴런은 닭, 식탁, 배, 식료품점처럼 **사물 전체**에 반응한다.

즉 층이 깊어질수록 뉴런이 반응하는 대상이 단순한 모양에서 점점 "고급" 정보로 바뀐다. 사물의 **의미**를 이해하는 방향으로 변하는 것이다. 이렇게 **단순한 특징을 조합하여 점점 추상적인 특징을 만드는 것**이 층을 깊게 쌓는 이유이다. 합성곱 계층을 여러 번 거칠수록 출력의 한 원소가 보는 입력의 범위(**수용 영역, receptive field**)가 넓어지므로, 뒤쪽 층일수록 이미지의 넓은 영역을 보고 판단할 수 있다.



(a) 필터 길이 3인 합성곱을 세 번 (1차원) (b) 같은 일을 이미지 (2차원)에서

뒤쪽 층으로 갈수록 넓어지는 수용 영역. 층 3의 원소 하나 (맨 위 주황색)가 입력 7칸에 의존한다.

3×3 필터 (스트라이드 1)를 L 번 쌓으면 수용 영역은 $(2L+1) \times (2L+1)$ 이 된다. 중간에 2×2 풀링 (스트라이드 2)이 들어가면 그 뒤의 한 칸이 입력의 두 칸에 해당하므로 수용 영역은 훨씬 빨리 넓어진다. 그래서 앞쪽 층은 작은 에지를, 뒤쪽 층은 이미지의 큰 부분이나 전체를 보고 판단할 수 있다.

7 대표적인 CNN

CNN의 원조인 LeNet과, 딥러닝이 주목받게 된 계기가 된 AlexNet을 살펴본다.

CNN 이전의 문자 인식: 사람이 설계한 특징

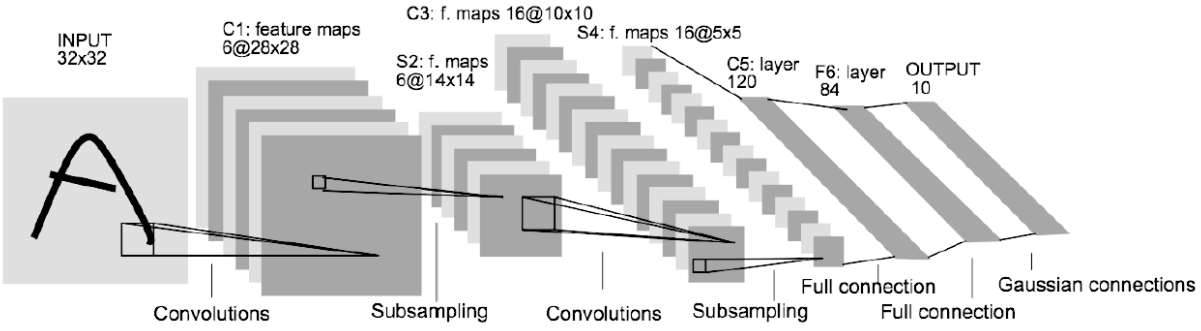
문자 인식(OCR)은 1950년대부터 패턴 인식의 대표 문제였고, 1990년대까지는 **"사람이 설계한 특징 추출기 + 학습하는 분류기"**가 주류였다.

- **템플릿 매칭 (1950–60년대):** 글자마다 표준 모양을 저장해 두고 가장 비슷한 것을 고른다 (템플릿 전체를 하나의 큰 필터로 쓴 셈). 위치, 크기, 글씨체가 조금만 달라도 실패하였다. 기계 판독을 쉽게 하는 **전용 글꼴**도 만들어졌다 (광학식 판독용 OCR-A, 수표의 자기잉크식 판독용 E-13B).
- **사람이 설계한 특징 (1960–90년대):** 방향 필터 (Sobel, Gabor)로 센 가로·세로·대각선 획의 양, 이미지를 구역으로 나누어 센 획의 양 (zoning, 사실상 평균 풀링), 행·열별 픽셀 수, 구멍·끝점·교차점의 개수, 이동과 크기에 변하지 않는 모멘트 등을 뽑아 k-NN, 통계적 분류기, SVM 등에 넣었다.
- **한계:** 문자 체계마다 전문가가 특징을 새로 설계하고 조정해야 했고, 손글씨의 다양한 기울기와 굵기에 약했으며, 특징 추출 단계에서 버린 정보는 분류기가 되살릴 수 없었다.

LeCun 등은 LeNet 논문 (1998)에서 바로 이 구조를 지적하고, **특징 추출기까지 데이터로 학습**하자고 제안하였다. 흥미롭게도 학습된 CNN의 첫 층 필터는 방향 에지 필터와 비슷해진다. 사람이 설계한 특징이 틀렸던 것이 아니라, 그 위의 복잡한 조합을 사람이 설계하기 어려웠던 것이다.

7.1 LeNet

LeNet은 손글씨 숫자를 인식하는 network로 1998년에 제안되었다 (LeCun 등). 합성곱 계층과 풀링 계층(정확히는 **서브샘플링** 계층: 2×2 영역을 더한 뒤 채널마다 학습하는 계수와 편향을 적용하고 활성화 함수를 통과시킨다)을 반복하고, 마지막으로 완전연결 계층을 거쳐 결과를 출력한다.



LeNet-5의 구조 (LeCun 등, 1998). "6@28 × 28"은 28 × 28 크기의 특징 맵 6장이라는 뜻이다.

그림의 C는 합성곱(Convolution), S는 서브샘플링(Subsampling), F는 완전연결 계층이다. C5는 5×5 입력에 5×5 필터 120개를 적용하여 1×1 출력 120개를 만들므로 사실상 완전연결 계층이다. 마지막 출력 (Gaussian connections)은 지금은 Softmax로 바뀌었다.

LeNet의 서브샘플링

서브샘플링(subsampling)은 "표본을 줄인다", 즉 해상도를 낮춘다는 뜻으로, LeNet의 풀링 계층이다. S2는 C1의 특징 맵마다 겹치지 않는 2×2 영역 하나에서 출력 하나를 만든다.

$$y = \sigma\big(w\,(x_1 + x_2 + x_3 + x_4) + b\big)$$

네 값을 더하고, 학습하는 계수 w를 곱하고 편향 b를 더한 뒤, 활성화 함수 σ (scaled tanh)를 통과시킨다. 예를 들어 영역이 $\begin{pmatrix} 1 & 3 \\ 2 & 6 \end{pmatrix}$ 이고 $w = 0.25$, $b = 0$이면 $0.25 \times 12 = 3$으로 네 값의 **평균**이다. 즉 서브샘플링은 **평균 풀링 (스트라이드 2)**에 학습하는 배율, 편향, 활성화를 덧붙인 것이고, 28×28 특징 맵이 14×14가 된다.

- **매개변수:** w와 b는 특징 맵마다 하나씩이고, 같은 맵의 모든 위치가 공유한다. S2는 $6 \times 2 = 12$개, S4는 $16 \times 2 = 32$개이다. 지금의 최대 풀링은 0개이다.
- **목적:** 손글씨는 쓸 때마다 획의 위치가 조금씩 다르므로, 특징의 정확한 위치는 오히려 방해가 된다. 해상도를 낮추면 위치가 조금 어긋나도 출력이 거의 같다. Hubel과 Wiesel의 복합 세포, Neocognitron의 C층과 같은 역할이다.

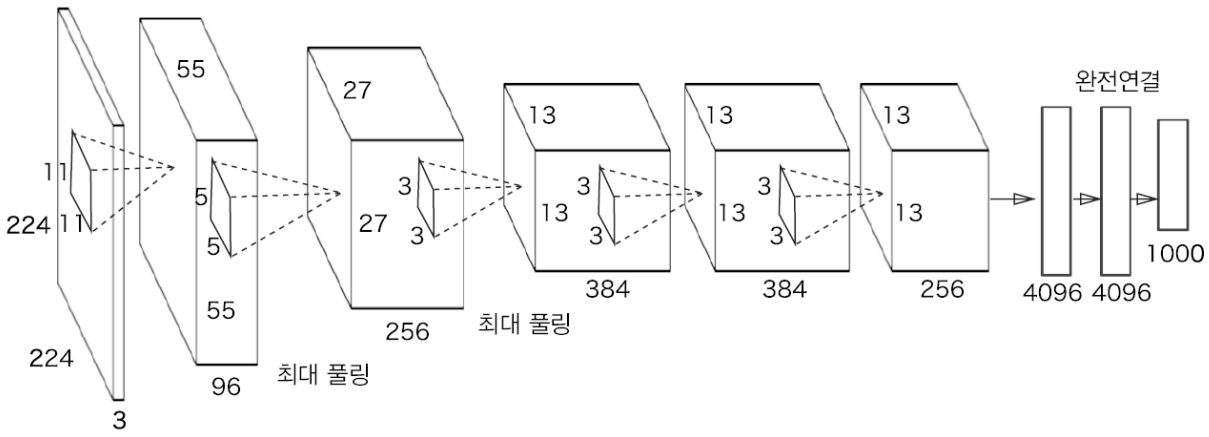
`SimpleConvNet`과 비교하면 거의 같은 구조이지만, 지금의 CNN과는 다음이 다르다.

- **활성화 함수**: LeNet은 sigmoid 계열의 함수 (scaled tanh)를 썼다. 지금은 주로 ReLU를 쓴다.
- **풀링**: LeNet은 매개변수가 있는 서브샘플링으로 중간 데이터의 크기를 줄였다. 지금은 매개변수가 없는 최대 풀링이 주류이다.

그래도 거의 30년 전에 이미 지금의 CNN과 같은 기본 구조가 완성되어 있었다는 것이 놀라운 점이다.

7.2 AlexNet

LeNet 이후 오랫동안 주목받지 못하던 CNN은 2012년 AlexNet (Krizhevsky, Sutskever, Hinton)의 등장으로 딥러닝 열풍을 일으켰다. 대규모 이미지 인식 대회(ILSVRC 2012)에서 top-5 오류율 약 15%로, 26%였던 2위와 큰 차이로 우승하였다.



AlexNet의 구조 (Krizhevsky 등, 2012): 합성곱 계층 5개와 완전연결 계층 3개

그림의 입력은 원 논문을 따라 224×224로 적혀 있지만, 첫 번째 층 (필터 11×11, 스트라이드 4)의 출력 55×55는 입력이 227×227이고 패딩이 0일 때 출력 크기 공식과 맞는다 (연습문제 1).

AlexNet은 합성곱 계층과 풀링 계층을 거듭하며 마지막으로 완전연결 계층을 거쳐 결과를 출력한다는 점에서 LeNet과 같지만, 다음과 같은 변화가 있었다.

- 활성화 함수로 **ReLU**를 쓴다.
- LRN(Local Response Normalization)이라는 국소적 정규화를 하는 계층을 쓴다 (지금은 배치 정규화로 대체되었다).
- **드롭아웃**을 쓴다 (완전연결 계층).
- 최대 풀링을 쓴다.

매개변수는 약 6000만 개이다. 이것을 학습할 수 있었던 것은 네트워크 구조뿐 아니라 **빅데이터** (ImageNet, 약 120만 장의 학습 이미지)와 대량의 병렬 계산을 하는 **GPU**가 보급된 덕분이다.

AlexNet에는 지금까지 배운 부품이 모두 있다

ReLU, 최대 풀링, 드롭아웃 (지난 주), 정규화 계층 (배치 정규화의 선배 격), GPU에서의 대규모 행렬곱 (im2col). 그 뒤의 CNN도 같은 부품을 더 깊게 쌓는 방향으로 발전하였다: VGG (2014, 3×3 합성곱을 쌓아 완전연결 계층 포함 16 ~ 19층), GoogLeNet (2014, 여러 크기의 필터를 병렬로), ResNet (2015, 입력을 건너뛰어 더하는 skip connection으로 150층 이상).

8 PyTorch로 구현하기

지난 시간에 PyTorch로 완전연결 network를 만들었다. 같은 방법으로 `SimpleConvNet`을 PyTorch로 만들면, 바뀌는 것은 계층의 이름뿐이다 (`mnist_cnn_torch.py`).

우리의 구현 (`conv_layers.py`)	PyTorch
`Convolution(W, B, stride, pad)`	`nn.Conv2d(C, FN, kernel_size, stride, padding)`
`Pooling(2, 2)`	`nn.MaxPool2d(2)`
`Flatten()`	`nn.Flatten()`
필터 `W`: (FN, C, FH, FW)	`weight`: (FN, C, FH, FW) (같은 순서)

```
model = nn.Sequential(
    nn.Conv2d(1, 30, kernel_size=5),      # (N, 1, 28, 28) -> (N, 30, 24, 24)
    nn.ReLU(),
    nn.MaxPool2d(2),                      # -> (N, 30, 12, 12)
    nn.Flatten(),                         # -> (N, 4320)
    nn.Linear(30 * 12 * 12, 100),
    nn.ReLU(),
    nn.Linear(100, 10),
).to(device)
```

데이터는 $(N, 1, 28, 28)$ 모양의 텐서로 만들어 넣는다. 학습 루프는 지난 시간의 코드와 같다.

```python
def to_dataset(x, t):
    """images (N, 784) -> (N, 1, 28, 28) tensors, one-hot labels -> class indices"""
    return TensorDataset(torch.tensor(x, dtype=torch.float32).reshape(-1, 1, 28, 28),
                         torch.tensor(np.argmax(t, axis=1)))
```

	검증 정확도					
	1 epoch	2 epoch	5 epoch	10 epoch	시험	epoch당
numpy (`conv_layers.py`)	95.4%	97.7%	98.6%	98.5%	98.5%	약 60초
PyTorch (CPU)	97.6%	98.4%	98.6%	99.1%	98.8%	약 12초

- 같은 구조, 같은 optimizer이므로 결과가 비슷하다. 초깃값과 미니배치 순서가 다르므로 숫자가 똑같지는 않다.
- 같은 CPU에서 PyTorch가 약 5배 빠르다. PyTorch는 합성곱을 C++로 최적화된 라이브러리로 계산하며, GPU가 있으면 `device="cuda"`만으로 더 빨라질 수 있다 (얼마나 빨라지는지는 환경에 따라 다르니 직접 재 보자).
- PyTorch의 `nn.Conv2d`는 기본 초깃값으로 균등분포 (범위 $\pm 1/\sqrt{25}$)를 쓴다. 학습된 필터에서 여러 방향의 에지 모양을 볼 수 있다.



PyTorch로 학습한 첫 번째 합성곱 계층의 필터 30개 (`model[0].weight`)

9 정리

합성곱 신경망: 정리

- CNN은 지금까지의 신경망에 **합성곱 계층**과 **풀링 계층**을 더한 것이다.
- 합성곱 계층은 이미지의 **형상을 유지**하며, 같은 필터를 모든 위치에 적용한다 (가중치 공유). 출력 크기는 $OH = \lfloor \frac{H+2P-FH}{S} \rfloor + 1$.
- 필터는 4차원 (FN, C, FH, FW), 데이터는 4차원 (N, C, H, W)이다.
- 풀링 계층은 매개변수 없이 공간 크기를 줄이며, 작은 위치 변화에 강하다.
- **im2col**을 쓰면 합성곱은 하나의 행렬곱, 즉 Affine 계층이 된다. 역전파는 Affine의 역전파와 col2im이다.
- 첫 층의 필터는 에지와 블롭에 반응하고, 층이 깊어질수록 더 추상적인 정보를 추출한다.
- LeNet (1998)과 AlexNet (2012)이 대표적인 CNN이며, 빅데이터와 GPU가 딥러닝 발전의 큰 원동력이었다.

10 연습문제

1. 다음 합성곱 계층의 출력 크기를 구하라. (a) 입력 $(32, 32)$, 필터 $(5, 5)$, 패딩 2, 스트라이드 1. (b) 입력 $(227, 227)$, 필터 $(11, 11)$, 패딩 0, 스트라이드 4 (AlexNet의 첫 번째 층). (c) 입력 $(28, 28)$에 2×2 최대 풀링 (스트라이드 2)을 두 번 적용한 결과.

2. 입력 $\begin{pmatrix} 1 & 0 & 2 & 1 \\ 0 & 3 & 1 & 0 \\ 2 & 1 & 0 & 4 \\ 1 & 0 & 1 & 2 \end{pmatrix}$ 에 필터 $\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$ 를 스트라이드 1, 패딩 0으로 적용한 결과를 손으로 계산하라. 같은 입력에 2×2 최대 풀링 (스트라이드 2, 패딩 0)을 적용한 결과도 구하라.

3. RGB 이미지 ($C = 3$)에 3×3 필터 64개를 쓰는 합성곱 계층의 매개변수(가중치와 편향) 수를 구하라. 입력이 $(3, 32, 32)$이고 패딩 1일 때, 같은 크기의 출력 $(64, 32, 32)$을 완전연결 계층으로 만든다면 가중치는 몇 개가 필요한가?

4. 2번 문제의 최대 풀링에서 출력 쪽 gradient가 $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ 일 때, 입력 쪽 gradient (4×4)를 구하라.

5. 길이 5인 1차원 입력 $(x_1, \ldots, x_5)$에 길이 3인 필터를 스트라이드 1로 적용하면 출력은 3개이다. im2col 행렬 (3×3)을 쓰고, x_3이 몇 번 등장하는지 세어 보라. 이로부터 col2im에서 gradient를 **더해야** 하는 이유를 설명하라.

6. SimpleConvNet의 합성곱 계층에 패딩 2를 주도록 고쳐 보라. `conv_out`, `pool_out`, `flat`은 각각 얼마가 되는가? Convolution 계층에 `pad`를 넘기는 부분과 `flat`의 계산을 고친 뒤, `conv_basics.py`의 gradient 확인으로 검증하라.

7. `mnist_cnn_torch.py`를 고쳐 LeNet과 비슷한 network (Conv 5×5 6개, 패딩 2 → ReLU → MaxPool → Conv 5×5 16개 → ReLU → MaxPool → FC 120 → ReLU → FC 84 → ReLU → FC 10)를 만들고 학습시켜라. 각 계층을 지난 뒤의 shape을 표로 정리하고, 매개변수 수와 검증 정확도를 SimpleConvNet과 비교하라.